

Unidade I

1 FUNDAMENTOS DE C#

C# é uma linguagem de programação criada pela empresa Microsoft. Ela serve para construir muitos tipos de software: sites, serviços em nuvem, programas de computador, aplicativos para celular e jogos. É moderna, com checagem de tipos (ajuda a evitar erros) e gerenciamento automático de memória, além de ter recursos de produtividade como `async/await` e LINQ. C# é a linguagem principal da plataforma .NET.

.NET é a plataforma que executa programas escritos em C#. A edição .NET 10 é o lançamento de 2025 (ciclo anual da Microsoft) e é um LTS (Long Term Support)

1.1 Sintaxe, tipos, nullability, var, organização de solução/projeto

A linguagem C# pertence à família das linguagens derivadas de C, possuindo uma sintaxe bastante familiar para quem já utilizou C, C++, Java ou JavaScript. Nela cada instrução termina com ponto e vírgula (;) e blocos de código são delimitados por chaves { e }. Os identificadores (nomes de variáveis, métodos, classes etc.) são case-sensitive, ou seja, diferencia maiúsculas de minúsculas, inclui as estruturas de controle usuais, como `if/else` para condicionais, `switch` para seleção de casos e laços de repetição `for`, `while` e `foreach`, com sintaxes muito semelhantes às do C/C++.

No código tradicional, começamos declarando um namespace (Exemplo) para organizar logicamente a classe Program. Dentro da classe, definimos o método Main com o modificador `static`, indicando que é um método de classe (e não de instância). O método Main sem parâmetros é convencionalmente o ponto de entrada de programas C# executáveis – ou seja, é a primeira função a ser invocada quando o programa é iniciado. Nele, fazemos a chamada `Console.WriteLine` para produzir saída no console.

C# é uma linguagem fortemente tipada, o que significa que cada variável e constante no código possui um tipo bem definido em tempo de compilação. Toda expressão em C# avalia para um tipo específico, conhecido pelo compilador, e operações envolvendo valores geralmente só são permitidas se os operandos forem de tipos compatíveis. Esse forte sistema de tipos promove a detecção precoce de erros: o compilador (ou mesmo as ferramentas do ambiente de desenvolvimento) emite erros ou avisos caso tentemos usar uma variável de modo inconsistente com seu tipo declarado.

Algumas dessas palavras-chaves são `int` (em C#), que corresponde ao tipo inteiro de 32 bits assinado (`System.Int32` na plataforma .NET); da mesma forma, `double` é um tipo de ponto flutuante de 64 bits (`System.Double`); `char` representa um caractere Unicode de 16 bits (`System.Char`); `bool` é um valor booleano lógico (`System.Boolean`, que pode ser `true` ou `false`), e assim por diante. O quadro 1 ilustra alguns dos tipos primitivos e seus propósitos.

Tipos por valor são aqueles que armazenam diretamente seus dados. Isso significa que uma **variável de um tipo por valor contém em si o valor atribuído**, e não uma referência a um valor em outro lugar. Os **tipos numéricos primitivos (int, double etc.), bool, char e também os tipos definidos com struct (estruturas) são tipos por valor.** Por padrão, instâncias de tipos por valor são **alocadas na stack (pilha)** de execução ou embutidas dentro de estruturas de dados maiores. Uma consequência importante é que, ao atribuir uma variável de tipo por valor a outra, realiza-se uma cópia dos dados.

Tipos por referência, por outro lado, armazenam referências (endereços) para objetos alocados na heap (área de memória dinâmica). Class (classes), arrays (vetores) e string (cadeia de caracteres) são exemplos de tipos por referência. Quando declaramos uma variável de um tipo por referência, **inicialmente ela não aponta para nenhum objeto concreto e seu valor padrão é null (nulo)**, indicando nenhuma referência válida. **Para obter um objeto real, geralmente usamos o operador new para instanciar a classe, o que aloca memória na heap e retorna uma referência para essa região de memória.** A variável então armazena esse ponteiro (de forma segura, sem aritmética de ponteiros explícita na linguagem gerenciada). **Ao atribuir uma variável de referência a outra, não copiamos os dados do objeto em si, mas sim a referência (o endereço).**

Obs: Instanciar a classe significa criar um objeto concreto a partir de uma classe

Outra característica dos tipos por referência é que eles podem representar a ausência de um valor real através de null. Uma variável de referência não inicializada (ou explicitamente setada como null) indica que ela não aponta para nenhum objeto válido naquele momento; tentar usar (desreferenciar) uma referência nula em tempo de execução causa a exceção `System.NullReferenceException`

Em memória de programas, **stack e heap** descrevem **estratégias distintas de organizar dados** ao longo da execução. **A primeira se assemelha a uma pilha** de pratos: tudo que entra por cima sai por cima, em ordem estritamente **last-in, first-out (LIFO)**. **A segunda lembra um almoxarifado: há espaço para itens de tamanhos variados, guardados e recuperados conforme a necessidade**, sem seguir a mesma rigidez de ordem.

A pilha serve de suporte imediato à execução de funções. Quando um procedimento começa, o sistema cria um quadro de ativação que costuma reunir parâmetros, variáveis locais e o endereço de retorno; ao término, esse quadro é descartado integralmente. Como contrapartida, o espaço é limitado e o padrão LIFO impõe vidas curtas e bem delimitadas aos dados; empilhar estruturas grandes ou encadear chamadas profundas pode resultar em estouro de pilha, e dados com tamanho muito variável ou que precisam persistir além da execução da função não são adequados para esse tipo de armazenamento.

O heap, por sua vez, atende a dados cujo tamanho ou duração não cabem nas amarras da pilha. Estruturas dinâmicas, coleções que crescem com a entrada do usuário e objetos que atravessam fronteiras de funções são alocados nessa área. Em ambientes com GC, como a plataforma .NET, o runtime monitora a alcançabilidade dos objetos e libera a memória quando deixam de ser referenciados; em linguagens sem coleta automática, cabe

ao programador desalocar no momento correto. A flexibilidade tem custo: a alocação exige metadados e coordenação do alocador, e a liberação depende do coletor ou de disciplina manual. Além disso, referências mantidas desnecessariamente podem causar retenção de memória

A pilha privilegia velocidade e previsibilidade ao preço de limites rígidos; o heap oferece maleabilidade com sobrecusto de gerenciamento.

Tipos por valor são eficientes e evitam alocação dinâmica quando contêm dados pequenos e imutáveis, mas não podem ser nulos por padrão. Tipos por referência acomodam estruturas de dados mais complexas e de tamanho variável, permitindo compartilhamento de objetos, ao custo de ser preciso gerenciar (indiretamente) o fato de que múltiplas referências podem apontar para o mesmo objeto.

Comparações de igualdade (== em tipos por valor compara conteúdo, enquanto em tipos por referência compara referência, exceto se o operador for sobrecarregado) ou questões de gerenciamento de memória

Um exemplo como `int x = 5;` ilustra o caso comum de um valor simples associado ao escopo imediato da função: variáveis locais de tipos por valor tendem a residir na pilha durante a execução, o que favorece acesso rápido e descarte automático ao final do bloco. A documentação apresenta justamente essa intuição ao discutir como reduzir alocações ao preferir **struct** (um tipo de valor que pode encapsular dados e funcionalidades relacionadas.) em cenários adequados.

Quando a instrução envolve `new`, como em `var pessoa = new Pessoa();`, a instância resultante é criada no heap gerenciado, enquanto a variável local retém apenas uma referência a esse objeto. Essa separação entre a variável que contém o valor (tipo por valor) e a variável que contém uma referência (tipo por referência) é um ponto central da especificação da linguagem.

No ecossistema .NET, a memória do heap é administrada pelo GC. O runtime acompanha a alcançabilidade dos objetos e, quando não existem mais referências ativas, sua memória se torna elegível para liberação; o processo é automático e poupa o desenvolvedor de desalocação manual, evitando erros clássicos de uso após liberação. Para casos estritamente temporários e com tamanho conhecido, a linguagem oferece ainda `stackalloc`, que reserva um bloco na pilha e o descarta integralmente ao retorno do método, fora do escopo do GC.

Os tipos anuláveis por valor (nullable value types). Usando a sintaxe de interrogação após o tipo – por exemplo, `int?` em vez de `int` –, podemos declarar um tipo por valor que aceite representar também a ausência de valor através de `null`. Em termos técnicos, `int?` é um complemento sintático para `System.Nullable`, uma estrutura genérica que encapsula um valor do tipo especificado e um flag indicando se há um valor presente ou não. Assim, `int? x = null;` é válido e inicializa `x` sem um valor inteiro concreto.

Esse recurso foi criado para lidar com situações como campos de banco de dados ou informações opcionais que possam ser desconhecidas ou indisponíveis, aplicando-se apenas a tipos por valor, já que tipos por referência sempre puderam ser nulos.

Se tentamos atribuir null a uma variável que foi declarada como de tipo não anulável (por exemplo, fazer nome2 = null onde nome2 é string não anulável), o compilador emite um aviso ou um erro conforme a configuração, pois estamos violando a promessa de não nulidade, se uma variável é não anulável, o compilador espera que ela seja inicializada com um valor não nulo antes de ser usada.

Para suprimir intencionalmente um aviso desses (quando se tem certeza do estado não nulo), C# fornece o operador de null-forgiveness !. Por exemplo, poderíamos escrever string texto = mensagem!; para indicar ao compilador: “confie em mim, mensagem não é nula aqui”. Isso remove o aviso, porém deve ser usado com cautela: é uma forma de sobrescrever o mecanismo de análise estática e, se usado incorretamente (quando a variável for de fato nula), o erro de runtime continuará existindo. Idealmente, deve-se preferir checagens explícitas de null (if (mensagem != null) { ... }) antes de utilizar a variável, ou empregar o operador de coalescência nula (??) para fornecer valores alternativos em caso de null. Nos bastidores, string e string? são do mesmo tipo System.String no runtime, e a distinção existe apenas para o compilador.

A nulabilidade no C# moderno envolve duas frentes: (1) os tipos por valor anuláveis (T?, em que T é um valor, via Nullable), que existem desde C# 2.0 e permitem representar null para structs e primitivos; e (2) os tipos por referência anuláveis (habilitados em C# 8+), que introduzem análise estática para evitar usos inadvertidos de referência nula.

Em C#, declaramos variáveis especificando primeiramente o tipo, seguido do nome da variável e, opcionalmente, de uma atribuição inicial.

Var só pode ser utilizado em variáveis locais (dentro de métodos ou blocos de código). Ele não é permitido em campos de classe ou estrutura, nem em parâmetros de métodos.

Use var quando o tipo do lado direito é óbvio para um leitor humano ou quando é muito verboso. Por exemplo, ao escrever var lista = new List();, é bastante claro que lista será do tipo List, eliminando a repetição do tipo em ambas as partes da declaração. Por outro lado, em algo como var numero = ObterValor();, no qual o tipo retornado por ObterValor() pode não ser evidente, talvez seja melhor especificar o tipo explicitamente para melhorar a legibilidade, a menos que o contexto deixe claro do que se trata.

Em situações de tipos anônimos, var é obrigatório. O C# permite criar objetos ad hoc com propriedades sem definir uma classe formal – os chamados tipos anônimos. Por exemplo: var anon = new { Nome = “Maria”, Idade = 30 };. Aqui, anon terá um tipo anônimo gerado pelo compilador (sem nome acessível no código), então não haveria como declarar essa variável de outra forma senão com var. Em consultas LINQ, por exemplo, muitas vezes obtemos sequências de objetos anônimos, e nesses casos usar var é necessário para manipular os resultados.

Exemplificando o cenário de tipos anônimos e var em conjunto, veja este trecho:

```
var resultado = from c in clientes where c.Cidade == "São Paulo" select new {  
    NomeCliente = c.Nome, AnoNascimento = c.DataNascimento.Year };  
  
foreach (var item in resultado) {  
    Console.WriteLine($"Cliente: {item.NomeCliente}, Nascido em: {item.AnoNascimento}");  
}
```

Em C# no ecossistema .NET, o código-fonte é escrito dentro de projetos, e esses projetos são agrupados em uma solução. A solução (arquivo .sln) é um contêiner lógico: ela guarda a lista de projetos, as dependências entre eles, as configurações de build e os perfis de execução. Cada projeto (arquivo .csproj) define como compilar um artefato (uma biblioteca, um aplicativo de console, uma Web API, um conjunto de testes). Essa separação permite evoluir módulos de forma independente e reutilizar bibliotecas em diferentes aplicativos.

Dito de outro modo, o código é organizado em projetos (.csproj) que geram artefatos específicos – bibliotecas reutilizáveis, aplicativos de console, serviços web – e em uma solução (.sln) que agrega esses projetos, registra as dependências entre eles e guarda configurações de compilação e execução. A solução funciona como índice da aplicação e dita a ordem de build com base nas referências declaradas; cada projeto, por sua vez, define o que compilar, com quais opções e quais outras unidades de código consome.

1.2 Ferramentas: Visual Studio 2026 e dotnet CLI (estrutura .csproj, SDK-style)

O Visual Studio 2026 é a versão mais recente da IDE da Microsoft, lançada em novembro de 2025 e identificada internamente como versão 18, com foco em integração profunda de inteligência artificial e no suporte nativo a .NET 10 e C# 14, além de apresentar melhorias de desempenho em relação ao Visual Studio 2022.

A edição Visual Studio Community 2026 é a porta de entrada gratuita. A própria Microsoft a descreve como uma IDE “free, fully-featured” para estudantes, contribuidores de projetos open source e desenvolvedores individuais, conjunto de recursos centrais de edição de código, depuração, testes, integração com Git e suporte a .NET 10, C# 14, C++ e demais workloads modernos é o mesmo da linha paga, com restrições concentradas no tipo de organização e no número de usuários que podem utilizar tal versão em cenários comerciais: organizações não empresariais, com até cinco usuários, podem adotar o Community de forma legítima, enquanto grandes empresas precisam migrar para o Professional ou o Enterprise. Git é um sistema de controle de versão distribuído usado para

acompanhar o histórico de alterações em arquivos, especialmente código-fonte. Ele permite trabalhar com branches, mesclar contribuições de vários desenvolvedores e manter um repositório completo em cada máquina.

```
using System;

Console.WriteLine("Olá! Vamos testar a sintaxe básica do C# 14.");

// Tipos e variáveis
int ano = 2026;
var nome = "Ana";
Console.WriteLine($"Boas-vindas, {nome}. Estamos em {ano}.");

// Nulabilidade e operadores relacionados
string? apelido = null;
var exibir = apelido ?? "visitante";
Console.WriteLine(exibir); // "visitante"
apelido ??= "convidado"; // atribuí só se for nulo
Console.WriteLine(apelido);

// Controle de fluxo e pattern matching com expressão switch
object? x = 150;
string descricao = x switch
{
    null => "nulo",
    int n when n > 100 => "inteiro grande",
    string s => $"texto com {s.Length} chars",
    _ => "desconhecido"
};
Console.WriteLine(descricao);

// Métodos locais e expressão de corpo
static decimal Calcular(decimal preco, decimal taxa = 0.12m)
    => preco * (1 + taxa);
Console.WriteLine(Calcular(100m)); // 112,00

// Records (dados com igualdade estrutural) e with-expressions
var p1 = new Produto(1, "Café", 15m);
var p2 = p1 with { Preco = 18m };
Console.WriteLine(p2);

// Propriedade com o token contextual 'field' (C# 14)
var cfg = new Config { Tema = "claro" };
Console.WriteLine(cfg.Tema);

// Atribuição condicional com ?. à esquerda (C# 14)
Cliente? cliente = new Cliente();
cliente?.Total += 100m; // só avalia se cliente != null
Console.WriteLine(cliente?.Total);

public record Produto(int Id, string Nome, decimal Preco);
public class Cliente { public decimal Total { get; set; } }
public class Config
{
    public string Tema
    {
        get;
        set => field = value ?? throw new ArgumentNullException(nameof(value));
    }
}
```

Não há um Main explícito, porque o compilador gera nos bastidores um ponto de entrada e executa as linhas na ordem em que aparecem, desde que declarações de tipos (como classes) venham depois desses comandos.

Declaração `string? apelido = null;` indica que a variável pode ser nula e faz o analisador estático alertar sobre acessos potencialmente inseguros

O coalescente nulo ?? devolve o operando da esquerda quando não é nulo e, caso contrário, o da direita; além disso, a análise de fluxo “afunila” o tipo do resultado para não anulável quando o alternativo é não anulável, como em `var exibir = apelido ?? “visitante”;`. Já `??=` atribui apenas se o destino estiver nulo: `apelido ??= “convidado”;` preenche uma lacuna sem sobrescrever um valor já existente.

Em `object? x = 150;`, o literal inteiro é convertido para `object` (boxing) e armazenado numa referência que também pode ser nula. A variável é então analisada por uma expressão `switch`, que é uma construção que avalia padrões e produz um valor. Cada braço contém um padrão seguido de `=>`. O primeiro testa o padrão constante `null`; o segundo verifica simultaneamente o tipo e captura o valor com um padrão de tipo (`int n`) e ainda impõe uma guarda `when n > 100`; o terceiro reconhece `string` e usa a interpolação para incluir `s.Length`; o curinga `_` é o descarte para qualquer outro caso.

Trata-se de uma função local marcada como `static`, o que sinaliza que ela não captura variáveis do escopo externo. A sintaxe do corpo da expressão usa a seta para indicar que o valor de retorno é a expressão à direita. O parâmetro `taxa` tem valor padrão (`0.12m`), e o sufixo `m` especifica o tipo decimal, adequado a cálculos monetários por evitar erros de representação binária comuns ao tipo `double`. A saída exata de `Console.WriteLine(Calcular(100m));` depende de formatação e da cultura; com a formatação padrão, zeros à direita podem não ser exibidos. A linha `public record Produto(int Id, string Nome, decimal Preco);` define um record de classe com forma posicional. O compilador gera propriedades públicas imutáveis por inicialização (`init`), construtor, métodos de deconstrução e igualdade baseada no conteúdo (dois registros `Produto` com os mesmos componentes são iguais).

A expressão `p1 with { Preco = 18m };` cria um novo objeto `Produto` copiando todos os componentes de `p1` e alterando apenas o valor de `Preco` para `18m`. O `p1` permanece inalterado com `Preco = 15m`. O `p1` permanece inalterado com `Preco = 15m`, enquanto o `p2` é outra instância.

A classe `Config` mostra uma autopropriedade com validação no acessor `set`. O acessor de leitura `get`; é automático e o acessor de escrita usa corpo de expressão. Dentro do `set`, `value` é o valor recebido na atribuição e `field` é um token contextual que referencia o campo de apoio gerado para a propriedade automática. A atribuição condicionada `field = value ?? throw new ArgumentNullException(nameof(value));` impede que `Tema` receba `null`, lançando uma exceção que usa `nameof(value)` para registrar o identificador do argumento. A criação `new Config { Tema = “claro” };` usa inicializador de objeto, que chama o construtor padrão e, em seguida, atribui às propriedades, acionando a validação do `set`.

A sequência com `Cliente? cliente = new Cliente();` e `cliente?.Total += 100m;` ilustra acesso condicional. O operador `?.` só avalia o membro seguinte se o receptor não for nulo; quando aplicado a uma atribuição composta, a operação só acontece quando existe um destino válido e, se o receptor for nulo, nada é executado. A leitura `cliente?.Total` segue a mesma regra e resulta em `null` quando o receptor é nulo.

Em C#, namespaces são agrupamentos lógicos de tipos relacionados, algo análogo a pastas que guardam arquivos. O namespace `System` concentra muitos tipos básicos da biblioteca padrão, entre eles o tipo `Console`, que representa a tal janela de texto.

A diretiva `using System;` serve para dizer ao compilador que queremos usar esses tipos sem precisar escrever o nome completo. Assim, em vez de escrever `System.Console.WriteLine(...)` a cada vez, basta digitar `Console.WriteLine(...)`. Em seguida, o código não define explicitamente um método `Main`; ele usa as chamadas instruções de nível superior. Isso significa que podemos escrever comandos diretamente no arquivo e o compilador gera, nos bastidores, um ponto de entrada equivalente a um `Main` tradicional.

Métodos são blocos de código que podem receber entradas e realizar alguma ação; nesse caso. Uma variável é um espaço na memória que recebe um nome e pode armazenar um valor. Aqui, `apelido` e `nome` são variáveis. Na linha `string? apelido = null;`, a variável `apelido` é declarada como `string?`. O ponto de interrogação indica que essa variável pode assumir o valor especial `null`, que significa “não há valor”.

```
var exibir = apelido ?? "visitante"; Console.WriteLine(exibir);
```

A leitura intuitiva é: se `apelido` não for `null`, use o valor de `apelido`; se for `null`, use “visitante”.

```
apelido ??= "convidado"; // atribui só se for nulo
```

```
Console.WriteLine(apelido);
```

Aqui, a lógica é: se `apelido` estiver `null`, atribua “convidado”; caso contrário, deixe o valor atual como está. É um modo compacto de preencher um valor padrão sem sobrescrever algo que já foi definido.

mistura tipos por valor, tipos por referência e o tipo base `object`: `object? x = 150;`.

```
string descricao = x switch
{
    null => "nulo",
    int n when n > 100 => "inteiro grande",
    string s => $"texto com {s.Length} chars",
    _ => "desconhecido"
};
Console.WriteLine(descricao);
```

Essa expressão avalia `x` e, para cada ramo do `switch`, verifica se o valor se encaixa (“casa”) no padrão descrito. O primeiro braço compara com o literal `null`: se `x` for `nulo`, o resultado é a string “nulo”. O segundo ramo usa um padrão de tipo com guarda: `int n` verifica se `x` é do tipo inteiro; se for, armazena o valor em `n`, e a condição `when n > 100` impõe que esse inteiro seja maior que 100. Nessa situação, o resultado será “inteiro grande”. O terceiro ramo do `switch` testa se `x` é do tipo `string` (`string s`); se for, captura em `s` e usa outra string interpolada para informar o número de caracteres, por meio de `s.Length`. Por fim, o último ramo usa `_`, que funciona como um curinga: corresponde a qualquer valor que não tenha sido tratado pelos casos anteriores, e produz a string “desconhecido”.

A sintaxe `=>` é um corpo de expressão: indica que o valor de retorno é exatamente o resultado da expressão à direita. Seria equivalente a escrever um método com bloco entre chaves e uma instrução `return`.


```

public record Produto(int Id, string Nome, decimal Preco);
public class Cliente { public decimal Total { get; set; } }
public class Config
{
    public string Tema
    {
        get;
        set => field = value ?? throw new ArgumentNullException(nameof(value));
    }
}

```

Em C#, tanto record quanto class definem tipos por referência, isto é, tipos cujas variáveis armazenam referências para objetos que vivem na memória gerenciada.

Se você criar dois objetos Cliente com os mesmos valores em suas propriedades, por padrão o C# considera que são objetos diferentes; a igualdade padrão de uma class é por referência, isto é, duas variáveis só são iguais se apontarem para a mesma instância na memória, a menos que alguém sobrescreva manualmente os métodos de comparação.

Um record, por outro lado, é voltado para dados. A ideia é que o foco está nos valores que ele carrega, não na identidade da instância.

A expressão `p1 with { Preco = 18m }`; cria um novo objeto Produto copiando todos os componentes de p1 e alterando apenas o valor de Preco para 18m. O p1 permanece inalterado com Preco = 15m, enquanto o p2 é outra instância.

```

public class Cliente { public decimal Total { get; set; } }

```

Uma classe é um tipo definido pelo programador que descreve um molde ou modelo para certos dados e comportamentos. Os atributos (em C#, normalmente representados por campos e propriedades) guardam o estado do objeto, enquanto os métodos representam ações ou operações que podem ser executadas sobre esse estado.

Essa classe define o tipo Cliente com uma propriedade Total do tipo decimal. O par `get; set;` indica uma propriedade automática: o compilador cria um campo interno para guardar o valor, além dos métodos de leitura e escrita.

Quando se escreve `public decimal Total { get; set; }`, isso significa que Total é uma propriedade com acessor de leitura (get) e de escrita (set). O formato `{ get; set; }`, nesse caso, é uma propriedade automática: o compilador entende que você quer guardar um valor simples, cria um campo escondido para armazenar esse valor e gera os dois acessores de forma padrão, usando esse campo interno. Ao ler `cliente.Total`, na prática, o código chama o acessor `get`; ao escrever `cliente?.Total += 100m`.

A linguagem C# vive dentro de uma plataforma maior, chamada .NET (um runtime que implementa a especificação CLI, acompanhado de uma biblioteca padrão, sobre os quais linguagens como C#, Visual Basic e F# trabalham.), composta de um runtime e de bibliotecas. O .NET Framework é uma plataforma dividida em componentes: em primeiro lugar, o Common Language Runtime (CLR), que é o ambiente de execução virtual

responsável por executar o código gerenciado; em segundo lugar, a biblioteca de classes – Base Class Library (BCL)

O CLR é um runtime utilizável por diferentes linguagens, e C# é apenas uma delas. Em outras palavras, C# fica acima do CLR: ela determina a sintaxe e as regras da linguagem; o resultado da compilação é alimentado ao runtime de .NET

A BCL é o elo que faz C# e as demais linguagens .NET compartilharem o mesmo alicerce de tipos e APIs. Ela é um conjunto de bibliotecas fundamentais da plataforma, intimamente ligado ao Common Type System e ao runtime, além de ser usado por todas as linguagens que visam a CLI.

A linguagem C# é definida formalmente pela especificação ECMA-334 (ECMA International, 2023). Ela não é o .NET, mas sim uma linguagem que compila para a infraestrutura de linguagem comum (CLI, do inglês Common Language Infrastructure): o código-fonte em C# é compilado para um código intermediário (CIL, do inglês Common Intermediate Language) e metadados compatíveis com a CLI, que serão executados pelo CLR.

Runtime é o conjunto de componentes que fazem o programa ganhar vida na hora de executar. O arquivo do programa, sozinho, é só texto ou código gravado em disco. Quando você pede para executar uma aplicação .NET, o runtime entra em ação: carrega o programa na memória, traduz o código para instruções que o processador entende, administra o uso de memória, cuida da execução passo a passo e encerra tudo de forma organizada quando o programa termina.

Biblioteca, nesse contexto, é um conjunto de funcionalidades prontas, empacotadas em arquivos de código que o desenvolvedor pode reaproveitar. Em vez de cada programador ter que escrever do zero tudo o que um programa faz (por exemplo, mostrar texto na tela, acessar arquivos, conversar com a internet, trabalhar com datas e números grandes e assim por diante), essas tarefas comuns ficam reunidas em bibliotecas.

SDK é a sigla para Software Development Kit, ou kit de desenvolvimento de software. A ideia é parecida com um kit de ferramentas que você compra para realizar um tipo de trabalho específico. É um pacote que reúne tudo de que um desenvolvedor precisa para criar programas para uma determinada plataforma ou tecnologia.

O dotnet CLI ocupa o lugar de ferramenta fundamental de linha de comando. A documentação oficial o descreve como uma interface multiplataforma para desenvolver, compilar, executar e publicar aplicações .NET

O dotnet CLI organiza-se em torno do executável dotnet, que funciona como uma espécie de driver ou despachante de comandos. Ao invocar dotnet no terminal, seguido de um verbo como new, build ou run, o driver carrega o SDK instalado, interpreta os argumentos e delega o trabalho aos compiladores e ferramentas apropriados.

O Visual Studio 2026 é uma IDE que integra de forma nativa .NET 10 e C# 14, fornecendo editor, depuração, designers e integração com serviços de nuvem, mas ele se

apoia justamente no SDK do .NET – que, por sua vez, inclui a CLI – para compilar e executar os projetos. Assim, o mesmo projeto que é aberto na IDE pode ser manipulado inteiramente pelo terminal, a CLI fornece uma superfície estável que permite alternar entre ambiente gráfico e script de automação sem alterar o código-fonte.

CLI comandos => Comandos como `dotnet build`, `dotnet run`, `dotnet test` e `dotnet publish` conduzem, respectivamente, à compilação, à execução, à execução de testes automatizados e à geração de artefatos prontos para distribuição.

Pipelines de integração contínua são sequências automáticas de passos que executam em um servidor sempre que alguém altera o código de um sistema. Em vez de um desenvolvedor se lembrar de compilar, executar testes, verificar se os pacotes foram restaurados e, às vezes, preparar o que será publicado, tudo isso é descrito em um roteiro (pipeline). Esse roteiro é executado de forma automática a cada alteração enviada ao repositório de código. A ideia é detectar problemas cedo: se uma mudança quebrou a compilação, falhou em testes ou trouxe uma dependência incompatível, o pipeline acusa o problema logo após o envio do código.

Servidores de build são máquinas (físicas ou na nuvem) dedicadas a executar esses pipelines. Em vez de compilar apenas no computador pessoal de cada desenvolvedor, a organização usa um ou mais servidores configurados especificamente para isso. Nesses servidores, comandos como `dotnet restore` e `dotnet build` são executados de maneira previsível, em um ambiente controlado.

CI/CD é a sigla para continuous integration/continuous delivery (ou continuous deployment, dependendo do contexto). Continuous integration é justamente essa prática de integrar alterações ao repositório principal com frequência, usando pipelines automatizados de compilação, restauração de dependências e testes em servidores de build. Continuous delivery/deployment amplia essa automação até as etapas de entrega: depois que o código passa por todas as verificações, o mesmo pipeline pode preparar versões instaláveis, publicar pacotes, atualizar um site ou um serviço em produção, tudo com o mínimo possível de intervenção manual. Dizer que a CLI pode ser acionada por scripts em servidores CI/CD significa que os mesmos comandos `dotnet` que o desenvolvedor digita na própria máquina são utilizados dentro desses pipelines automatizados.

CLI pode ser visto como o núcleo textual do desenvolvimento com .NET 10 e C# 14: é ele que concretiza, em comandos simples, as operações essenciais de criação, restauração, compilação, teste e publicação de projetos, servindo tanto de base técnica para ferramentas gráficas como o Visual Studio 2026 quanto de interface direta para scripts e terminais em ambientes diversos.

2 POO: PILARES E ENCAPSULAMENTO NA PRÁTICA

(POO) é um paradigma de desenvolvimento que modela o software como um conjunto de objetos, cada um combinando estado (atributos) e comportamento (métodos).

Uma classe é uma descrição abstrata de um conjunto de coisas que têm características e comportamentos em comum. É como um modelo: nela o programador especifica quais dados existirão (os atributos ou campos) e quais ações poderão ser executadas (os métodos).

Entre esses métodos, em linguagens orientadas a objetos como C#, existe uma categoria especial chamada método **construtor**. O construtor é executado automaticamente quando um novo objeto da classe é criado e é usado para inicializar os atributos e estabelecer o estado inicial necessário para que esse objeto seja utilizado de maneira consistente. Em C#, o construtor tem o mesmo nome da classe, não declara tipo de retorno e pode receber parâmetros.

Um objeto, por sua vez, é uma realização concreta de uma classe durante a execução do programa. Quando o programa é executado e o código solicita a criação de algo com base em uma classe, o que surge na memória é um objeto. Esse objeto carrega valores específicos para os atributos definidos na classe e é capaz de executar os métodos que a classe descreve.

```
class Carro
{
    public string Cor { get; set; }
    public string Modelo { get; set; }
    public int Ano { get; set; }

    // Método construtor
    public Carro(string cor, string modelo, int ano)
```

64

PROGRAMAÇÃO ORIENTADA A OBJETOS COM C#

```
{
    Cor = cor;
    Modelo = modelo;
    Ano = ano;
}

public void Acelerar() { /* ... */ }
public void Frear() { /* ... */ }
}
```

Nesse código, Carro é a classe: ela define quais dados cada carro terá (Cor, Modelo, Ano) e quais operações podem ser feitas (Acelerar, Frear). O trecho `public Carro(string cor, string modelo, int ano)` é o método construtor.

```
var carroDoAluno = new Carro("Azul", "Hatch", 2022);
```

o que surge na memória é um objeto concreto, `carroDoAluno`, isto é, uma instância da classe `Carro`. Esse objeto possui valores específicos para os atributos definidos na classe (cor Azul, modelo Hatch, ano 2022) e passa a responder aos métodos declarados, de modo que chamadas como `carroDoAluno.Acelerar()`; e `carroDoAluno.Frear()`; utilizam o comportamento descrito na definição de `Carro`.

De forma sintética, o dot operator em C# é o operador que conecta nomes e valores em cadeias de acesso: ele entra em namespaces, alcança tipos e, a partir deles ou de instâncias, alcança campos, propriedades e métodos, inclusive métodos de extensão. A partir do momento em que a expressão à esquerda não é nula e o membro à direita existe e é acessível.

Uma convenção didática amplamente adotada em livros técnicos é descrever a POO a partir de quatro mecanismos centrais – encapsulamento, abstração, herança e polimorfismo –, frequentemente chamados de “os quatro pilares” da orientação a objetos.

O primeiro pilar, o encapsulamento, diz respeito ao empacotamento de dados e operações que atuam sobre esses dados em uma unidade lógica, normalmente uma classe, isolando a representação interna e expondo apenas uma interface bem definida.

Textos clássicos sobre POO com C++ e C# descrevem encapsulamento como a reunião de atributos e funções em uma mesma unidade, associada ao princípio de data hiding: o código externo não acessa diretamente o estado interno, mas interage por meio de métodos ou propriedades

Em C#, isso se concretiza em decisões como declarar campos privados, expor propriedades com `get/ set` controlados, usar modificadores de acesso (`public`, `protected`, `internal`, `private`) e definir métodos que representam operações conceitualmente relevantes sobre o objeto.

A abstração, segundo a mesma literatura, é o processo de selecionar os aspectos essenciais de uma entidade e ignorar detalhes contingentes, de modo que um tipo represente um conceito coerente do domínio de problema.

Em C#, classes e interfaces são os principais instrumentos de abstração: uma interface pode capturar um conjunto de operações que caracterizam, por exemplo, um repositório ou um serviço de notificação, sem impor como essas operações serão implementadas; classes concretas então realizam essa abstração em termos de campos, algoritmos específicos e recursos da plataforma.

A herança, terceiro pilar, é descrita nas referências como o mecanismo pelo qual uma classe (subclasse ou classe derivada) reaproveita e especializa a estrutura e o comportamento de outra (superclasse ou classe base). No material específico de C#, enfatiza-se que todas as classes derivam, em última instância, de `System.Object`, e que a

linguagem oferece herança simples de classes combinada com implementação múltipla de interfaces, permitindo construir hierarquias em que tipos mais gerais capturam propriedades comuns, enquanto tipos mais específicos acrescentam comportamentos ou restringem usos.

Polimorfismo é a capacidade de diferentes objetos responderem à mesma mensagem de formas específicas, preservando uma interface comum. Em textos didáticos sobre C# e POO, o polimorfismo é descrito como a situação em que duas ou mais classes oferecem o mesmo conjunto

2.1 Encapsulamento, abstração, herança e polimorfismo: visão geral

Encapsular significa agrupar dados e métodos relacionados em uma única unidade (a classe), ocultando os detalhes internos e expondo apenas o necessário para o uso externo, a classe define quais operações estão disponíveis para o mundo exterior (sua interface pública) enquanto protege seus dados internos e como essas operações são realizadas (sua implementação).

Encapsulamento pode ser definido como => “o processo de compartimentalizar os elementos de uma abstração que constituem sua estrutura e comportamento; esse processo serve para separar a interface contratual de uma abstração de sua implementação”, somente as partes relevantes para o uso da classe são expostas, enquanto detalhes internos ficam inacessíveis, evitando dependências indevidas.

Escondemos os dados e a lógica de implementação por trás de barreiras bem definidas (geralmente limites de classe ou módulo), expondo apenas o que for necessário por meio de métodos ou propriedades públicas, atributos de um objeto geralmente ficam marcados como privados, inacessíveis diretamente de fora, e somente podem ser consultados ou modificados por meio de operações definidas na interface pública desse objeto.

Exemplo: classe Pessoa com um atributo público idade. Qualquer código externo poderia livremente atribuir um valor inválido a idade (como um número negativo ou muito acima do plausível), deixando o objeto em um estado inconsistente. Já com o encapsulamento, podemos tornar idade um campo privado e fornecer um método ou uma propriedade pública para alterá-lo, o qual rejeitaria ou ajustaria valores inválidos.

Encapsular reduz o acoplamento entre diferentes partes do programa. Objetos interagem apenas com as interfaces de outros objetos, sem precisar conhecer (nem depender de) detalhes internos uns dos outros. Isso leva a uma arquitetura mais modular, em que mudanças internas em uma classe não afetam outras classes desde que a interface pública permaneça a mesma.

Exemplo de encapsulamento:

```
public class ContaBancaria
{
    private decimal saldo; // campo privado

    public decimal Saldo // propriedade pública somente leitura
    {
        get { return saldo; }
        private set { saldo = value; }
    }

    public ContaBancaria(decimal saldoInicial = 0m)
    {
        saldo = saldoInicial;
    }

    public bool Depositar(decimal quantia)
    {
        if (quantia <= 0)
            return false;
        saldo += quantia;
        return true;
    }

    public bool Sacar(decimal quantia)
    {
        if (quantia <= 0 || quantia > saldo)
            return false;
        saldo -= quantia;
        return true;
    }
}
```



Campo interno saldo é privado, e a classe fornece uma propriedade pública somente leitura Saldo (com get público e set privado) para permitir consulta externa do saldo, mas não modificação arbitrária

Cabe observar que encapsulamento e abstração são conceitos complementares que andam de mãos dadas. A abstração lida com o que um objeto representa e o que ele oferece – foca o comportamento observável do objeto, omitindo complexidades desnecessárias. Já o encapsulamento lida com como esse comportamento é implementado – oculta a complexidade interna que produz o comportamento.

2.1.2 Abstração

Abstração é o processo de destacar as características essenciais de um objeto, omitindo detalhes supérfluos ou irrelevantes para determinado contexto.

POO podemos criar uma classe abstrata Veiculo que captura essa essência genérica, enquanto classes concretas como Carro ou Motocicleta estendem Veiculo e implementam detalhes particulares (como o número de rodas e o tipo de combustível, entre outros). Nesse exemplo, Veiculo é uma abstração que reúne as características comuns, servindo como modelo para as especializações.

a abstração em POO tem dois significados complementares. O primeiro é conceitual: trata-se de modelar entidades do mundo real ou conceitos de forma simplificada, incluindo apenas atributos e operações pertinentes ao contexto da aplicação. O segundo significado é técnico: refere-se aos mecanismos da linguagem que nos permitem esconder detalhes de implementação e enfatizar comportamentos gerais.

Em C# e em outras linguagens orientadas a objetos modernas, a abstração se manifesta principalmente através de classes abstratas e interfaces, a abstração é o ato de se concentrar nos aspectos fundamentais de uma entidade e ignorar quaisquer detalhes que não sejam relevantes naquele contexto

Em C#, definimos uma classe abstrata usando a palavra-chave `abstract`. Uma classe abstrata é aquela que não pode ser instanciada diretamente; ela existe principalmente para ser herdada por outras classes. Normalmente, uma classe abstrata representa um conceito genérico ou uma generalização, e suas subclasses representam implementações específicas desse conceito.


```

// Classe abstrata representando um Animal genérico
public abstract class Animal
{
    public string Nome { get; set; }

    // Construtor da classe abstrata
    public Animal(string nome)
    {
        Nome = nome;
    }

    // Método abstrato: cada subclasse deve implementar o seu próprio som
    public abstract void EmitirSom();
}

// Subclasse concreta que estende Animal
public class Cachorro : Animal
{
    public Cachorro(string nome) : base(nome) { }
    // Implementação específica do som de um cachorro
    public override void EmitirSom()
    {
        Console.WriteLine($"{Nome}: Au Au!");
    }
}

// Outra subclasse concreta de Animal
public class Gato : Animal
{
    public Gato(string nome) : base(nome) { }

    // Implementação específica do som de um gato
    public override void EmitirSom()
    {
        Console.WriteLine($"{Nome}: Miau!");
    }
}

// Programa principal para testar as classes anteriores
public class Program
{
    public static void Main()
    {
        Animal meuAnimal1 = new Cachorro("Rex");
        Animal meuAnimal2 = new Gato("Felix");

        // Mesmo sem saber exatamente o tipo, podemos chamar EmitirSom graças à
        // abstração
        meuAnimal1.EmitirSom(); // Saída: "Rex: Au Au!"
        meuAnimal2.EmitirSom(); // Saída: "Felix: Miau!"
    }
}

```

O pilar da abstração em POO nos incentiva a pensar nas entidades de um programa em termos do que elas representam e do que podem fazer, antes de mergulhar em como elas fazem. Abstraindo corretamente, delineamos fronteiras conceituais claras para nossas classes e interfaces, o que conduz a um design mais limpo e compreensível.

2.1.3 Herança

É o mecanismo que permite definir uma nova classe a partir de uma classe existente, reaproveitando atributos e comportamentos já implementados na classe base.

A classe existente cuja definição é reutilizada é chamada de classe base (ou superclasse), enquanto a nova classe definida que herda essas características é chamada de classe derivada (ou subclasse).

Por exemplo, considere uma hierarquia simples em que a classe Veiculo define atributos e métodos genéricos (como cor, ano, acelerar(), frear()), e uma classe derivada Carro herda de Veiculo. O Carro automaticamente possui todas as propriedades e comportamentos definidos em Veiculo, como cor e acelerar(), sem precisar reescrevê-los. Além disso, Carro pode introduzir características específicas (como um atributo tipoCombustivel ou um método abrirPorta()). Assim, Carro é um Veiculo, estabelecendo uma relação do tipo é-um (is-a) própria da herança.

Em C#, a herança de classes é implementada especificando-se a superclasse após dois-pontos na declaração da subclasse. Cada classe em C# pode ter no máximo uma classe base direta – ou seja, o C# adota herança simples (single inheritance).

```
class Animal {
    public string Nome { get; set; }
    public void Comer() {
        Console.WriteLine($"{Nome} está comendo.");
    }
    public virtual void FazerSom() {
        Console.WriteLine("Som de animal genérico");
    }
}
class Cachorro : Animal {
    public void AbanarRabo() {
        Console.WriteLine($"{Nome} está abanando o rabo.");
    }
    public override void FazerSom() {
        Console.WriteLine("Au Au!");
    }
}
```

É importante ressaltar que, se a classe Animal não declarasse o método FazerSom() como virtual, não seria possível sobrescrevê-lo corretamente em Cachorro – qualquer tentativa de declarar um método FazerSom() em Cachorro sem a correspondência virtual resultaria em um novo método independente (ocultando o da superclasse). O C# até permite essa ocultação deliberada usando o modificador new, mas tal prática não fornece polimorfismo real e geralmente não é recomendada, pois pode confundir a compreensão do código. Para aproveitar o polimorfismo proporcionado pela herança, os métodos que devem ter comportamento especializado nas subclasses precisam ser marcados como virtual na superclasse (ou abstract, caso a classe base queira apenas definir a assinatura e exigir implementação pelas derivadas) e então sobrescritos com override nas subclasses.

Vale notar também que membros públicos da superclasse permanecem acessíveis como públicos na subclasse; membros declarados como protected na superclasse também são herdados e podem ser utilizados dentro da subclasse (como se fizessem parte dela), mas não ficam acessíveis para outras classes fora da hierarquia. Já os membros private da superclasse não são visíveis na subclasse – eles existem apenas no escopo interno da própria classe base.

2.1.4 Polimorfismo

Em essência, o polimorfismo descreve a capacidade de um mesmo componente ou referência de assumir diferentes formas de comportamento. Ou seja, uma mesma interface ou chamado de método pode resultar em ações diferentes, dependendo do objeto concreto com o qual está trabalhando. Costuma-se resumir essa ideia na frase “uma interface, múltiplas implementações” – indicando que um único conjunto de operações (interface) pode desencadear métodos distintos conforme a classe específica do objeto em questão.

Em C#, por exemplo, uma classe pode ter múltiplos métodos com o mesmo nome, desde que possuam assinaturas diferentes (isto é, diferentes tipos ou quantidade de parâmetros). O compilador distingue qual versão do método chamar analisando os argumentos fornecidos em cada chamada. Assim, métodos sobrecarregados apresentam o mesmo nome, porém comportamentos diferentes de acordo com o contexto de chamada, realizando o princípio de “uma interface, múltiplas implementações” em nível de compilação.

Para ilustrar o polimorfismo em tempo de compilação, considere a classe Calculadora a seguir.

```
using System;

class Calculadora {
    // Método Somar para somar dois números inteiros
    public int Somar(int a, int b) {
        return a + b;
    }

    // Método Somar para "somar" (concatenar) duas strings
    public string Somar(string a, string b) {
        return a + b;
    }
}

class Programa {
    static void Main() {
        Calculadora calc = new Calculadora();
        // Chamando Somar com inteiros - invoca a primeira versão do método
        int resultadoNumerico = calc.Somar(5, 3);
        Console.WriteLine("Soma de inteiros: " + resultadoNumerico); // Saída:
        "Soma de inteiros: 8"

        // Chamando Somar com strings - invoca a segunda versão do método
        string resultadoTexto = calc.Somar("Olá, ", "Mundo!");
        Console.WriteLine("Soma de strings: " + resultadoTexto); // Saída:
        "Soma de strings: Olá, Mundo!"
    }
}
```

O outro tipo de polimorfismo, e geralmente o mais destacado nas discussões, é o polimorfismo em tempo de execução, também conhecido como polimorfismo dinâmico ou late binding. Esse é o polimorfismo no sentido mais clássico em orientação a objetos: a habilidade de um objeto de uma classe derivada de ser tratado como um objeto de sua classe base, mas executar automaticamente a versão sobrescrita de um método que for específica de sua classe real, o mesmo método invocado através de uma referência a um tipo base pode ter múltiplas implementações diferentes, escolhidas de acordo com o tipo concreto do objeto em tempo de execução.

Para que o polimorfismo dinâmico funcione, geralmente entram em cena dois conceitos intimamente ligados: herança (ou implementação de interfaces) e sobrescrita de métodos (method overriding).

Isto é, a classe base deve declarar o método com a palavra-chave virtual (ou ser um método abstract) para indicá-lo como passível de sobrescrita, e a classe derivada usa a palavra-chave override para sobrescrever a implementação.

```
using System;

// Classe base Animal com um método virtual
class Animal {
    public virtual void FazerSom() {
        Console.WriteLine("Som de animal indefinido.");
    }
}

// Subclasse Cachorro, derivada de Animal, sobrescrevendo o método FazerSom
class Cachorro : Animal {
    public override void FazerSom() {
        Console.WriteLine("O cachorro faz: Au au!");
    }
}

// Subclasse Gato, derivada de Animal, sobrescrevendo o método FazerSom
class Gato : Animal {
    public override void FazerSom() {
        Console.WriteLine("O gato faz: Miau!");
    }
}

class Programa {
    static void Main() {
        // Podemos criar objetos das subclasses:
        Cachorro dog = new Cachorro();
        Gato cat = new Gato();

        // Podemos tratá-los como seu tipo específico:
        dog.FazerSom(); // Saída: "O cachorro faz: Au au!"
        cat.FazerSom(); // Saída: "O gato faz: Miau!"

        // Porém, mais importante, podemos referenciar objetos de subclasses
        usando uma variável do tipo da classe base:
        Animal animal1 = new Cachorro();
        Animal animal2 = new Gato();

        // Chamando o método FazerSom() nas variáveis do tipo base:
        animal1.FazerSom(); // Saída: "O cachorro faz: Au au!"
        animal2.FazerSom(); // Saída: "O gato faz: Miau!"

        // Os objetos ainda são Cachorro e Gato por baixo dos panos, mas o
        programa pode tratá-los genericamente como Animal.
        // Graças ao polimorfismo em tempo de execução, cada chamada invoca a
        implementação correspondente à classe real do objeto.

        // Também é possível armazenar diversos objetos derivados em uma
        estrutura de dados do tipo da classe base:
        Animal[] animais = { new Cachorro(), new Gato() };
        foreach (Animal animal in animais) {
            animal.FazerSom();
            // Para cada elemento, a implementação específica de FazerSom() é
            chamada conforme o tipo real (Cachorro ou Gato).
        }
    }
}
```

2.2 Encapsulamento na prática: propriedades, invariantes, imutabilidade com record

O encapsulamento é um dos pilares da POO, caracterizado pela capacidade de esconder os detalhes internos de implementação de um objeto e expor apenas uma interface segura para uso externo. Em essência, o encapsulamento visa proteger os dados de uma classe contra acessos indevidos ou inconsistentes, permitindo que apenas métodos ou propriedades controlados manipulem seu estado, fornece uma barreira de proteção em torno do estado do objeto, evitando acessos não autorizados e possibilitando impor regras para alterações.

```
public class Pessoa {
```

```

private int idade; // campo encapsulado
public int Idade {
    get { return idade; }
    set
    {
        if (value < 0)
            throw new ArgumentOutOfRangeException("Idade não pode ser negativa.");
        idade = value;
    }
}

```

Nesse código, qualquer tentativa de atribuir um valor negativo a Idade resultará em uma exceção, impedindo a violação do invariante de que idade ≥ 0 . O uso da propriedade faz com que códigos externos manipulem a idade apenas através do nosso filtro (o acessor set), mantendo o objeto Pessoa sempre em um estado válido. Caso idade fosse um campo público acessado diretamente, nada impediria atribuições inválidas como `objPessoa.idade = -5`, possivelmente propagando erros difíceis de rastrear. encapsulamento garante que toda mudança de estado passe por uma verificação central, facilitando a manutenção e a evolução. Por exemplo, se futuramente decidirmos representar a idade em meses em vez de anos, podemos alterar a implementação interna mantendo a propriedade Idade – o código cliente continuaria usando Idade da mesma forma

Por exemplo, podemos definir um record assim:

```
public record Pessoa(string Nome, int Idade);
```

Em uma única linha, criamos uma classe imutável Pessoa com duas propriedades (Nome e Idade). O compilador gerará uma propriedade `public string Nome { get; init; }` e uma propriedade `public int Idade { get; init; }`, além de um construtor que recebe Nome e Idade como parâmetros. As propriedades marcadas com `init` só podem ser atribuídas durante a construção do objeto (seja no construtor ou em um inicializador de objeto logo depois), mas tornam-se somente leitura após o objeto estar pronto.

Assim, `Pessoa p = new Pessoa("Ana", 30);` cria um objeto cuja propriedade Idade vale 30, e nenhuma parte do código poderá alterá-la depois – não existe método set público. Qualquer tentativa de `p.Idade = 31` resultaria em erro de compilação, garantindo imutabilidade.

Do ponto de vista didático, é útil entender encapsulamento, invariantes e imutabilidade como partes de um mesmo esforço: o de controlar e preservar o estado válido dos objetos. Encapsular com propriedades nos permite interceptar e validar alterações de estado, enquanto definir objetos imutáveis elimina a possibilidade de alteração após a criação. Em ambos os casos, estamos evitando que o objeto assuma configurações inválidas.

Unidade II

3 ABSTRAÇÃO E INTERFACES

Sem um uso consciente de abstração, encapsulamento e contratos, o código tende a acumular dependências implícitas, trechos duplicados e suposições frágeis sobre o comportamento das classes.

C# oferece mecanismos específicos para estruturar essas decisões: classes abstratas e membros abstratos permitem modelar conceitos gerais que não fazem sentido, como objetos concretos, concentrando ali o código compartilhado e delegando às subclasses a responsabilidade por especializações obrigatórias.

As interfaces aparecem como outro pilar dessa arquitetura, agora enfatizando contratos puros e independentes de qualquer hierarquia concreta. Ao definir apenas assinaturas de métodos, propriedades e outros membros, as interfaces permitem que tipos de linhagens distintas assumam capacidades comuns e sejam tratados de forma uniforme por algoritmos polimórficos.

3.1 Classes e membros abstratos, ocultação de detalhes e contratos

No contexto da programação em C#, a abstração se manifesta por meio de classes abstratas e membros abstratos, enquanto o encapsulamento diz respeito à ocultação de detalhes de implementação.

Conceito de contrato aparece principalmente nas interfaces, que definem conjuntos de funcionalidades que as classes devem prover. Esses elementos trabalham em conjunto para produzir códigos mais modularizados, flexíveis e de fácil manutenção, ao separar nitidamente o “o quê” do “como” em uma aplicação.

Uma classe abstrata é aquela que não pode ser instanciada diretamente. Ela existe principalmente para servir de modelo ou classe base a outras classes mais especializadas. Uma classe abstrata frequentemente contém definições de métodos abstratos, os quais também não possuem implementação e devem obrigatoriamente ser substituídos (override) pelas subclasses concretas.

Uma classe abstrata pode definir tanto membros abstratos quanto membros concretos (não abstratos). Membros abstratos (que incluem métodos, propriedades, indexadores ou eventos) são declarados com o modificador `abstract` e não possuem corpo; já os membros não abstratos possuem implementação normal.

Classes abstratas podem inclusive definir campos e construtores. Embora não se possa criar instâncias de uma classe abstrata, suas subclasses podem invocar o construtor base (da classe abstrata) para reutilizar lógica de inicialização comum.

```
// Classe abstrata base
public abstract class Forma
{
    public string Nome { get; protected set; }
    // Construtor da classe abstrata base
    protected Forma(string nome) {
        Nome = nome;
    }

    // Membro abstrato: nenhuma implementação aqui
    public abstract double CalcularArea();
    // Método concreto: implementado na classe abstrata
    public void Descrever()
    {
        Console.WriteLine($"Esta forma é um {Nome}.");
    }
}

// Subclasse concreta da forma
public class Circulo : Forma
{
    private double raio;
```

PROGRAMAÇÃO ORIENTADA A OBJETOS COM C#

```
public Circulo(double raio) : base("circulo") {
    this.raio = raio;
}

// Implementação obrigatória do método abstrato
public override double CalcularArea()
{
    return Math.PI * raio * raio;
}

// Outra subclasse concreta
public class Retangulo : Forma
{
    private double largura, altura;

    public Retangulo(double largura, double altura) : base("retângulo") {
        this.largura = largura;
        this.altura = altura;
    }

    public override double CalcularArea()
    {
        return largura * altura;
    }
}
```

Forma é uma classe abstrata que define o contrato de que toda forma geométrica deve saber calcular sua área (CalcularArea). As subclasses Circulo e Retangulo herdam de Forma e implementam o método abstrato com as fórmulas específicas para área de círculo e retângulo, respectivamente. Observe que Forma provê também um método concreto Descrever(), que imprime uma descrição genérica usando um atributo comum (Nome).

A classe expõe apenas o que for necessário através de membros públicos (sua interface pública) e restringe o acesso às partes internas, marcando-as como privadas ou protegidas. No exemplo, os atributos raio, largura e altura nas subclasses são privados, logo, o código externo não pode modificá-los diretamente, garantindo integridade. Se for preciso ler ou modificar esses valores, a classe poderia fornecer métodos ou propriedades públicas controladas.

Em C#, uma interface é um tipo completamente abstrato, declarado com a palavra-chave `interface`, que contém definições de métodos, propriedades, eventos ou indexadores, mas sem qualquer implementação concreta.

```
public interface IForma
{
    double CalcularArea();
}
```

A interface `IForma` declara apenas a assinatura do método `CalcularArea`, sem implementação. Uma classe que implemente `IForma` deverá fornecer o corpo de `CalcularArea`. Se declararmos:

```
public class Triangulo : IForma
{
    public double Base { get; set; }
    public double Altura { get; set; }

    public Triangulo(double b, double h) {
        Base = b;
        Altura = h;
    }

    public double CalcularArea() {
        return (Base * Altura) / 2.0;
    }
}
```

Agora `Triangulo` é uma classe concreta que implementa `IForma`. O compilador força a existência do método `CalcularArea()` com exatamente a assinatura definida na interface, caso contrário a classe não compila (em C#, se faltar implementar algum membro da interface, a classe permanece incompleta e será tratada como `abstract` pelo compilador). Uma vez que `Triangulo` implementa `IForma`, podemos utilizá-lo por meio do tipo da interface:

```
IForma fig = new Triangulo(3, 4);
Console.WriteLine( fig.CalcularArea() ); // Saída: 6
```

Classes abstratas fazem parte de uma hierarquia de herança, normalmente representam uma generalização de um grupo de classes relacionadas. Elas podem fornecer implementação comum (código reutilizável) e também definir membros abstratos a serem implementados nas derivadas. Entretanto, cada classe concreta só pode herdar de uma classe base (limitação da herança simples em C#). Isso significa que a escolha de usar uma classe abstrata força um relacionamento hierárquico, por exemplo, `Circulo` é uma `Forma`, `ContaPoupanca` é um tipo de `ContaBancaria` abstrata etc.

Já as interfaces não implicam relação hierárquica é-um, mas sim definem capacidades que podem ser agregadas a qualquer classe, independentemente de onde ela esteja na hierarquia, podemos ter uma interface `Imprimivel` que define um contrato para objetos que possam ser impressos em papel. Uma classe `Relatorio` que já herda de uma classe base `Documento` (hipotética) pode implementar `Imprimivel` para sinalizar que sabe se imprimir; ao mesmo tempo, uma classe completamente diferente, digamos `Fotografia`, que herda de `Imagem`, também poderia implementar `Imprimivel`. Não existe um parentesco entre `Relatorio` e `Fotografia`, elas compartilham apenas a capacidade comum definida pela interface. Assim, as interfaces permitem formas de polimorfismo horizontal, no qual classes distintas compartilham comportamentos através de contratos comuns, sem estarem na mesma cadeia de herança.

Até recentemente, havia diferenças claras em termos de implementação: interfaces em C# não podiam fornecer nenhum código, pois todos os métodos de uma interface eram implicitamente abstratos (sem corpo) e públicos. As classes que implementassem a interface tinham de fornecer todo o código. Classes abstratas, por sua vez, podiam ter métodos com implementação concreta, permitindo fatorar código comum.

Contudo, versões mais novas do C# introduziram alguns recursos adicionais às interfaces. A partir do C# 8, por exemplo, interfaces podem conter implementações padrão em métodos (usando a palavra-chave `default` em métodos na interface), assim como também podem declarar membros estáticos, e até membros abstratos estáticos (como em C# 11) para certos cenários avançados.

Ainda assim, existem limitações, como no caso de interfaces que não podem ter campos de instância para armazenar estado, apenas constantes ou, mais recentemente, propriedades estáticas `readonly`. Dessa forma, classes abstratas continuam sendo a ferramenta adequada quando se deseja compartilhar estado ou código interno entre classes relacionadas, enquanto interfaces são mais indicadas para definir apenas contratos que qualquer classe pode adotar.

3.2 Interfaces, implementação explícita, substituíbilidade (LSP) com exemplos

Uma interface define um contrato que classes ou structs devem satisfazer por meio da implementação de determinados membros (métodos, propriedades, indexadores ou eventos).

Diferentemente de classes abstratas, interfaces não possuem estado e tradicionalmente não contêm implementação, elas especificam apenas o que deve ser feito, mas não como.

```

interface IFormaGeometrica {
    double CalcularArea();
}

class Retangulo : IFormaGeometrica {
    public double Largura { get; set; }
    public double Altura { get; set; }

    public double CalcularArea() {
        return Largura * Altura;
    }
}

class Circulo : IFormaGeometrica {
    public double Raio { get; set; }

    public double CalcularArea() {
        return Math.PI * Raio * Raio;
    }
}

```

Nesse código, **Retangulo** e **Circulo** oferecem implementações distintas do método **CalcularArea()**. Agora, graças ao polimorfismo proporcionado pelas interfaces, podemos escrever um método que calcula a área total de diversas formas geométricas sem nos preocuparmos com as classes concretas de cada uma, como vemos a seguir:

```

double AreaTotal(IFormaGeometrica[] formas) {
    double soma = 0;
    foreach (IFormaGeometrica forma in formas) {
        soma += forma.CalcularArea();
    }
    return soma;
}

// Uso do método:
var formas = new IFormaGeometrica[] {
    new Retangulo { Largura = 5, Altura = 4 }, // área = 20
    new Circulo { Raio = 3 } // área ≈ 28.27
};
Console.WriteLine(AreaTotal(formas)); // Saída esperada ≈ 48.27

```

As implementações explícitas são úteis em alguns cenários específicos. Um caso comum é quando uma classe implementa múltiplas interfaces que possuem métodos homônimos ou semanticamente distintos.

```

interface IGreetingsEn {
    string Saudar();
}

interface IGreetingsFr {
    string Saudar();
}

class CumprimentoMulti : IGreetingsEn, IGreetingsFr {
    // Implementação implícita para inglês:
    public string Saudar() {
        return "Hello"; // Saudação em inglês por padrão
    }
    // Implementação explícita para francês:
    string IGreetingsFr.Saudar() {
        return "Bonjour"; // Saudação em francês somente via interface IGreetingsFr
    }
}

```

O LSP é o L do acrônimo SOLID, um conjunto clássico de princípios de design de software. Formulado originalmente por Barbara Liskov em 1988, o LSP propõe que, se S é um subtipo de T, então os objetos do tipo T em um programa podem ser substituídos por objetos do tipo S sem que seja necessário alterar o comportamento correto desse programa, qualquer instância de uma classe derivada deve poder ser usada no lugar de uma instância da classe base (ou interface).

Esse princípio está intimamente ligado à ideia de contrato: a classe derivada (ou classe que implementa uma interface) deve honrar todas as garantias e expectativas estabelecidas pela classe base (ou interface).

```
class Retangulo {
    public virtual void DefinirLargura(double larg) { Largura = larg; }
    public virtual void DefinirAltura(double alt)    { Altura = alt; }
    public double Largura { get; protected set; }
    public double Altura  { get; protected set; }
    public virtual double Area => Largura * Altura;
}
```

```
class Quadrado : Retangulo {
    public override void DefinirLargura(double larg) {
        base.DefinirLargura(larg);
        base.DefinirAltura(larg);
    }
    public override void DefinirAltura(double alt) {
        base.DefinirAltura(alt);
        base.DefinirLargura(alt);
    }
}
```

```
void AumentarRetanguloEmDobro(Retangulo r) {
    r.DefinirLargura(r.Largura * 2);
    r.DefinirAltura(r.Altura * 2);
    // Esperamos que a área agora seja quatro vezes maior:
    Console.WriteLine("Área quadruplicada: " + r.Area);
}
```

```
// Cenário de uso:
Retangulo meuRet = new Retangulo();
meuRet.DefinirLargura(5);
meuRet.DefinirAltura(10);
// Área inicial = 50
AumentarRetanguloEmDobro(meuRet);
// Área esperada = 200 (50 * 4) -> correto para Retangulo

Retangulo meuQuad = new Quadrado();
meuQuad.DefinirLargura(5);
meuQuad.DefinirAltura(10);
// Aqui, ao definir altura para 10, a largura também se torna 10 (Quadrado impõe
// lados iguais)
// Área inicial de meuQuad = 100 (10*10)
AumentarRetanguloEmDobro(meuQuad);
// O método AumentarRetanguloEmDobro presume área quadruplicada,
// mas para Quadrado a área resultante será 400 (pois os lados dobraram de 10
// para 20, área 20*20)
```

O código cliente `AumentarRetanguloEmDobro` fica quebrado para o subtipo `Quadrado`, violando claramente o princípio da substituíbilidade (LSP). Em outras palavras, `Quadrado` é ainda um `Retangulo` em termos de herança de tipos, mas não se comporta como um retângulo genérico do ponto de vista de quem usa, uma vez que não permite tratar largura e altura de forma independente. Esse exemplo ilustra que nem toda relação é-um na modelagem conceitual se traduz em uma herança que respeita o LSP.

=> LSP, tipos derivados ou implementações de interfaces devem preservar as expectativas comportamentais do tipo base. Isso inclui manter válidas todas as afirmações lógicas (invariantes) do tipo base e não fortalecer restrições. Por exemplo, se uma interface `IRepositorio` define um método `Remover(item)` que, em sua documentação, promete remover um item se este existir, então uma classe que implemente essa interface não deveria simplesmente lançar uma exceção não suportada ao tentar remover, pois isso quebraria a expectativa do usuário da interface.

⇒ uma classe só pode herdar uma única classe abstrata, mas pode herdar múltiplas interfaces.

4 HERANÇA E POLIMORFISMO

Em desenvolvimento de software, a POO costuma ser apresentada como uma solução para organizar sistemas complexos em termos de tipos, responsabilidades e colaborações.

4.1 Hierarquias, abstract/sealed, composição como alternativa

Em POO, é comum organizar as classes em hierarquias de herança para modelar relações do tipo é-um entre entidades. Por exemplo, em um jogo poderíamos ter uma classe base genérica `Personagem` da qual derivam classes específicas como `Guerreiro` ou `Arqueiro`.

Em outras palavras, a subclasse abre a caixa da classe base e aproveita seu código, podendo alterar comportamentos por meio de métodos sobrescritos (override). A herança facilita o polimorfismo: um método que recebe um `Personagem` pode aceitar tanto um `Guerreiro` quanto um `Arqueiro`.

Além disso, o mecanismo de herança simples do C#, como em Java, não permite herança múltipla de classes. Assim, cada classe pode ter apenas uma superclasse direta. Isso significa que, quando se tenta modelar um conceito que naturalmente teria múltiplos pais, é preciso recorrer a outras técnicas, como interfaces ou composição.

Uma classe abstrata é uma classe pensada para ocupar o topo ou o meio de uma hierarquia, definindo um conceito genérico que não pode ser instanciado diretamente.

```

using System;

abstract class Personagem {
    public abstract void Atacar();
}

class Guerreiro : Personagem {
    public override void Atacar() {
        Console.WriteLine("O guerreiro ataca com sua espada!");
    }
}

class Arqueiro : Personagem {
    public override void Atacar() {
        Console.WriteLine("O arqueiro dispara uma flecha!");
    }
}

class Program {
    static void Main() {
        Personagem jogador1 = new Guerreiro();
    }
}

```

Unidade II

```

        Personagem jogador2 = new Arqueiro();
        jogador1.Atacar(); // Saída: O guerreiro ataca com sua espada!
        jogador2.Atacar(); // Saída: O arqueiro dispara uma flecha!
    }
}

```

Em contraste com as classes abstratas (que servem apenas como base), a linguagem C# também permite marcar classes de forma a impedir que outras herdem delas. Usa-se a palavra-chave `sealed` para indicar que uma classe não pode ter subclasses. É equivalente à classe `final` da linguagem Java.

Por exemplo, se declararmos `sealed class Arqueiro : Personagem { ... }`, o compilador não permitirá derivar nenhuma nova classe de `Arqueiro`. O objetivo de uma classe selada é sinalizar que sua hierarquia termina ali, seja por motivos de design (a classe representa uma entidade específica que não deve ser estendida inadvertidamente) ou por motivos de segurança e desempenho.

`sealed class Arqueiro : Personagem { ... }`, o compilador não permitirá derivar nenhuma nova classe de `Arqueiro`.

Além de classes inteiras, C# permite selar métodos virtuais individuais. Uma subclasse pode sobrescrever um método virtual da base e marcá-lo como `sealed override`, indicando que mais nenhuma classe derivada poderá sobrescrevê-lo novamente. Essa técnica é menos comum, mas útil para congelar personalizações em um determinado nível da hierarquia. A composição de objetos é outra forma fundamental de reutilização e organização de código. Em vez de estabelecer uma relação hierárquica *é-um*, a composição estabelece uma relação *tem-um* (has-a). Isso significa que um objeto é construído a partir de outros objetos internos, delegando a eles parte de sua funcionalidade.

Diferentemente da herança, na composição não há relação de especialização, nem compartilhamento direto de implementação, já que cada classe composta mantém sua individualidade e expõe apenas o que for necessário via sua interface pública, costuma-se chamar a composição de reuso black-box (caixa-preta), pois o objeto complexo trata seus componentes como caixas-pretas, utilizando-os através de suas interfaces, sem se acoplar aos detalhes internos deles.

Objetos complexos podem ser construídos agregando objetos menores; essa agregação modela características do mundo real (por exemplo, um carro tem-um motor, tem-um chassi, tem-um sistema elétrico etc.), promovendo um design modular.

Uma diretriz clássica de design orientado a objetos, apresentada por Gamma et al. (2005), é preferir composição em vez de herança sempre que possível. Essa recomendação reconhece que a herança, embora poderosa, pode levar a sistemas rígidos e fortemente acoplados, enquanto a composição favorece maior flexibilidade e menor acoplamento.

A composição torna mais fácil modificar comportamentos em tempo de execução, trocando os objetos componentes, em comparação com a herança, que fixa o comportamento no momento da definição da subclasse.

A composição oferece uma alternativa elegante: em vez de subclasses para cada tipo de ataque, usamos objetos “arma” que podem ser acoplados ou desacoplados de um personagem em tempo de execução. O personagem continua sendo um objeto de uma única classe, mas seu comportamento de ataque varia conforme o objeto arma que ele possui no momento.

A seguir, reimplementamos o cenário usando composição. Criamos uma classe Arma (abstrata), da qual derivam Espada e Arco, representando diferentes tipos de arma com suas lógicas de ataque. A classe Personagem agora não precisa ser abstrata nem ter subclasses para cada tipo, ela tem um campo do tipo Arma e delega a ela a ação de atacar. Assim, trocar a arma do personagem muda seu ataque sem ser preciso trocar o objeto Personagem em si.

```

using System;

abstract class Arma {
    public abstract void Atacar();
}

class Espada : Arma {
    public override void Atacar() {
        Console.WriteLine("Golpe de espada!");
    }
}

class Arco : Arma {
    public override void Atacar() {
        Console.WriteLine("Disparo de flecha!");
    }
}

class Personagem {
    public Arma ArmaAtual { get; set; }
    public Personagem(Arma armaInicial) {
        this.ArmaAtual = armaInicial;
    }
    public void Atacar() {
        if (ArmaAtual != null)
            ArmaAtual.Atacar();
        else
            Console.WriteLine("Personagem desarmado não pode atacar.");
    }
}

class Program {
    static void Main() {
        Personagem heroi = new Personagem(new Espada());
        heroi.Atacar();           // Saída: Golpe de espada!
        heroi.ArmaAtual = new Arco();
        heroi.Atacar();           // Saída: Disparo de flecha!
    }
}

```

4.2 Despacho virtual: virtual/override, polimorfismo por interface

Como vimos, o polimorfismo refere-se à capacidade de tratar objetos diferentes de forma uniforme. Em C#, o polimorfismo se manifesta principalmente de duas formas: através de métodos virtuais (mecanismo de despacho virtual em hierarquias de herança) e através de interfaces (polimorfismo por interface).

No contexto de herança de classes, a linguagem C# oferece suporte ao polimorfismo através de métodos virtuais e do mecanismo de despacho virtual. Uma classe base pode declarar que determinado método é virtual, indicando que as classes derivadas podem sobrescrevê-lo para alterar seu comportamento. Um método virtual definido na classe base

possui, portanto, uma implementação padrão, possivelmente incompleta ou genérica, que poderá ser sobrescrita (override) por uma classe derivada. Toda classe derivada concreta é obrigada a fornecer uma implementação (override) para esse método.

Método não virtual é chamado de método selado (sealed) nesse contexto, pois sua implementação fica fechada para modificações polimórficas em herdeiros. Portanto, para permitir que um método seja substituível em subclasses, a classe base deve declará-lo como virtual. Já a classe derivada usa o modificador override para indicar que está fornecendo uma nova implementação para um método virtual herdado.

Outra restrição sintática é que métodos virtuais não podem ser declarados como privados nem estáticos, tampouco podem ser marcados como sealed na própria classe base, um método com override não pode alterar a assinatura do método virtual correspondente.

```
using System;

class Character {
    public string Name { get; set; }
    public Character(string name) {
        Name = name;
    }
    public virtual void Attack() {
        Console.WriteLine($"{Name} ataca de forma genérica.");
    }
}

class Warrior : Character {
    public Warrior(string name) : base(name) { }
    public override void Attack() {
        Console.WriteLine($"{Name} avança com sua espada, desferindo um golpe poderoso!");
    }
}

class Mage : Character {
    public Mage(string name) : base(name) { }
    public override void Attack() {
        Console.WriteLine($"{Name} lança uma bola de fogo ardente contra o inimigo!");
    }
}

// Código de demonstração
class Program {
    static void Main() {
        Character[] party = new Character[2];
        party[0] = new Warrior("Arus");
        party[1] = new Mage("Luna");
        foreach (Character member in party) {
            member.Attack();
        }
    }
}
```


Nesse exemplo, `Character` declara o método `Attack()` como virtual, permitindo que subclasses o sobrescrevam. `Warrior` e `Mage` herdam de `Character` e cada um implementa sua versão própria de `Attack()`. No método `Main()`, criamos um array de `Character` contendo instâncias de `Warrior` e `Mage`.

Note que cada linha de saída corresponde à implementação definida na respectiva subclasse, demonstrando o polimorfismo em ação. Esse mecanismo interno de despacho virtual é viabilizado pelo runtime do .NET através de tabelas de métodos virtuais (v-table). Do ponto de vista do desenvolvedor, basta marcar os métodos com `virtual/override` e usar referências da superclasse; o runtime se encarrega de invocar a versão adequada.

Além do polimorfismo baseado em herança de classes, o C# fornece também o polimorfismo por interface. Vimos que interfaces em C# são tipos especiais que definem um conjunto de membros (métodos, propriedades, indexadores, eventos etc.) que representam uma capacidade ou um comportamento, porém sem implementar nenhum deles – isto é, uma interface contém apenas as assinaturas dos membros, cabendo às classes ou às estruturas que a implementam fornecer o código desses membros, todos os membros de uma interface são implicitamente públicos e abstratos, por isso, na definição da interface, não se coloca modificador de acesso nem corpo nos métodos/propriedades.

Um ponto poderoso das interfaces é que elas permitem uma forma de herança múltipla de comportamento no C#, um dos critérios de escolha entre usar herança de classes abstratas e usar interfaces: quando se quer aproveitar código comum e há claramente uma relação é-um entre os tipos, pode-se usar uma classe base; mas, quando se quer apenas impor comportamentos comuns a tipos potencialmente não relacionados, interfaces são a escolha ideal.

```
using System;

interface IDamageable {
    void TakeDamage(int amount);
}
```

Unidade II

```
class Player : IDamageable {
    public string Name { get; set; }
    public int Health { get; private set; }
    public Player(string name, int health) {
        Name = name;
        Health = health;
    }
    public void TakeDamage(int amount) {
        Health -= amount;
        Console.WriteLine($"{Name} recebeu {amount} de dano. Saúde restante: {Health}");
        if (Health <= 0) {
            Console.WriteLine($"{Name} foi derrotado!");
        }
    }
}

class Barrel : IDamageable {
    public int Durability { get; private set; }
    public Barrel(int durability) {
        Durability = durability;
    }
    public void TakeDamage(int amount) {
        Durability -= amount;
        Console.WriteLine($"Barril sofreu {amount} de dano. Durabilidade restante: {Durability}");
        if (Durability <= 0) {
            Console.WriteLine("O barril quebrou em pedaços!");
        }
    }
}

// Código de demonstração
class Program {
    static void Main() {
        IDamageable[] targets = new IDamageable[2];
        targets[0] = new Player("Hero", 100);
        targets[1] = new Barrel(50);
        foreach (IDamageable target in targets) {
            target.TakeDamage(30);
        }
    }
}
```

Uma dúvida comum é: quando usar herança (com métodos virtuais) e quando usar interfaces?

A resposta depende do relacionamento conceitual entre os tipos e do reaproveitamento de código desejado. De modo geral, a herança de classes é adequada quando existe uma relação clara de generalização/especialização (como uma relação é-um, onde um Mage é um Character) e quando faz sentido prover alguma implementação padrão ou estado compartilhado na superclasse. Com a herança, podemos centralizar código comum na classe base e sobrescrever apenas diferenças nas subclasses, além de podermos armazenar estado (atributos) que todas as subclasses herdam.

Por outro lado, as interfaces são indicadas quando queremos definir um conjunto de operações que podem ser realizadas por objetos de classes completamente distintas, potencialmente sem nenhuma relação de herança entre si, ou mesmo pertencentes a hierarquias diferentes. Interfaces promovem um menor acoplamento – o código consumidor depende apenas da interface, e não de uma classe específica – e permitem a um tipo adquirir múltiplas capacidades sem quebrar o modelo de herança simples da linguagem.

unidade III

5 COLEÇÕES E GENÉRICOS

A interface `IEnumerable` e o mecanismo de iteração com `foreach` entram nesse cenário como uma camada de abstração: permitem tratar coleções distintas como sequências genéricas, esconder detalhes de armazenamento e até criar sequências virtuais com `yield return`, em que os valores são produzidos sob demanda em vez de serem todos materializados de uma vez.

A partir do momento em que o código passa a ser escrito de forma genérica, surge outra família de problemas, como: restringir os tipos aceitos para garantir que certas operações estarão disponíveis; expressar igualdade e ordenação de maneira consistente; e como fazer isso sem cair em verificações manuais frágeis em tempo de execução. É aí que entram as restrições com `where` e o uso de interfaces como `IEquatable`, `IComparable`, `IComparer` e `IEqualityComparer`, além de padrões de implementação que mantêm `Equals`, `GetHashCode` e `CompareTo` coerentes entre si.

5.1 List, Dictionary, iteração e IEnumerable

`List` e `Dictionary` são dois dos tipos de coleções genéricas mais comuns no C#. Ambas pertencem ao namespace `System.Collections.Generic`, que reúne coleções capazes de armazenar múltiplos objetos.

Tipos por valor, como `int`, `double`, `bool` e `DateTime`, são armazenados de forma diferente de tipos por referência, como `string` e classes em geral. Quando um valor desses é colocado em uma variável do tipo `object` ou em uma coleção que guarda um `object`, o runtime precisa embrulhar esse valor em um objeto alocado no heap: isso é o `boxing`. O `unboxing` é o caminho inverso: pegar um `object` que, na verdade, contém um valor empacotado, verificar se o tipo interno é compatível e, se for, copiar de volta para uma variável do tipo por valor correto. Por exemplo, ao adicionar um `int` a um `ArrayList`, ocorre `boxing`; ao recuperar e atribuir de volta a um `int` com algo como `int x = (int)lista[0];`, há `unboxing`. Esse processo tem custo de desempenho (alocação e cópia) e risco de erro em tempo de execução, caso o objeto não contenha o tipo esperado.

Já `cast` é o termo usado para conversão explícita de tipos em C#. Quando você escreve `(TipoDesejado)expressão`, está dizendo ao compilador para tratar tal valor como sendo de outro tipo, sujeito a verificações em tempo de execução. Em coleções não genéricas que armazenam `object`, toda vez que se recupera um elemento é comum precisar de um `cast` para o tipo concreto – por exemplo, `string nome = (string)arrayList[0];`. Se o objeto ali dentro não for de fato uma `string`, o programa lança uma `InvalidCastException`. Nas coleções genéricas (`List`, `Dictionary`), o parâmetro de tipo `T` já fixa o tipo dos elementos em tempo de compilação, de forma que cada item recuperado da coleção vem com o tipo correto, dispensando `casts` explícitos na maioria dos casos.

List armazena diretamente valores int, não há boxing nem unboxing ao adicionar ou ler esses elementos. É por isso que afirmamos há pouco que o uso de coleções genéricas evita unboxing ou cast desnecessário: ao especializar a coleção para um tipo concreto, o compilador garante a segurança de tipos, elimina conversões em tempo de execução e reduz o risco de erros por conversões inválidas.

List é projetada como um array cujo tamanho é aumentado dinamicamente conforme necessário, resolvendo esse problema. Na prática, ao criarmos List e adicionarmos itens a ela via List.Add(...), seu espaço interno se expande automaticamente. Quando a capacidade atual (propriedade Capacity) é ultrapassada pelo número de itens (propriedade Count), o List aloca um array maior e copia os elementos antigos, garantindo que sempre haja espaço

```
List numeros = new List();
numeros.Add(5);
numeros.Add(10); numeros.Add(15);
```

List realoca internamente; caso contrário, podemos reduzir o espaço chamando métodos como Trim() para igualar Capacity ao valor de Count.

List, utiliza-se normalmente o comando foreach. O C# permite que qualquer objeto que implemente IEnumerable seja iterado com essa estrutura. List já implementa IEnumerable.

```
foreach (int n in numeros)
{
    Console.WriteLine(n);
}
```

O foreach funciona nos bastidores chamando GetEnumerator() no objeto lista, obtendo um enumerador (IEnumerator) que percorre os elementos internamente.

Já Dictionary é uma coleção genérica de pares chave-valor, ou seja, mapeia cada chave a um valor correspondente. Conforme Gunnerson e Wienholt (2012) definem, Dictionary é uma coleção de chaves e valores. A chave (TKey) serve de índice único; não podem existir duas entradas com a mesma chave.

```
Dictionary idades = new Dictionary();
idades["Alice"] = 30;
idades["Bob"] = 25;

foreach (KeyValuePair par in idades)
{
    Console.WriteLine($"{par.Key}: {par.Value}");
}

foreach (string chave in idades.Keys) { Console.WriteLine(chave); }

foreach (int valor in idades.Values) { Console.WriteLine(valor); }
```

Note-se que Dictionary também implementa IEnumerable, bem como as interfaces não genéricas correspondentes. Isso significa que todo Dictionary é enumerável.

Assim, seria possível criar uma classe própria, como uma lista personalizada, implementando IEnumerable para torná-la compatível com foreach. O exemplo clássico é implementar IEnumerable retornando um IEnumerator através de um método GetEnumerator(). Além disso, o C# fornece o recurso iterator com yield return, que simplifica a implementação de IEnumerable sem escrever a classe enumeradora manualmente. Vejamos o exemplo a seguir:

```
public IEnumerable NumerosImpares(int n)
{
    for(int i = 1; i <= n; i += 2) yield return i;
}
```

5.2 Genéricos: restrições, igualdade/ordenação e boas práticas

Genéricos são um recurso central do C# para escrever código que funciona com vários tipos mantendo segurança de tipos e boa performance. Em vez de criar uma classe, método ou interface específica para cada tipo concreto, declaramos um parâmetro de tipo, como T, e deixamos o compilador verificar se o uso é consistente.

Para que esse parâmetro de tipo não seja completamente livre, a linguagem oferece restrições genéricas com a palavra-chave where, que permitem dizer ao compilador que T precisa ser um tipo por referência ou um tipo por valor, implementar uma interface específica, herdar de uma classe base ou possuir um construtor público sem parâmetros. Dessa forma, é possível escrever código genérico que ainda assim faz suposições fortes sobre o tipo usado, com verificação em tempo de compilação em vez de verificações manuais em tempo de execução.

Também é possível impor que T derive de uma classe base específica, por exemplo where T : MinhaClasseBase, garantindo que todos os T tenham pelo menos os membros definidos nessa base. Por fim, where T : IMinhaInterface obriga T a implementar determinada interface, habilitando chamadas diretas aos métodos da interface sem casts.

Classe declarada como estática, isto é, como um tipo que não pode ser instanciado com new, não pode ser herdado e contém apenas membros estáticos. Em C#, uma classe estática (falamos brevemente sobre ela anteriormente) atua como um contêiner de funções auxiliares sem estado, deixando explícito que não existe identidade de objeto nem dados de instância a serem mantidos.

6 LINQ ESSENCIAL

O LINQ (Language-Integrated Query) é uma das tecnologias mais novas e poderosas de desenvolvimento da Microsoft, que integra consultas diretamente nas

linguagens .NET, como o C#. O nome é dado ao conjunto de recursos que integra operações de consulta diretamente à linguagem, permitindo escrever consultas sobre coleções, bancos de dados, XML e outras fontes usando sintaxe tipada em C# em vez de strings soltas.

Do ponto de vista técnico, o LINQ se apoia em duas peças centrais: as chamadas query expressions (a sintaxe from... where... select... embutida na linguagem) e os standard query operators, que são métodos de extensão definidos principalmente nas classes System.Linq.Enumerable e System.Linq.Queryable.

Esses operadores atuam sobre sequências que implementam IEnumerable ou IQueryable e oferecem operações como filtragem, projeção, ordenação, agrupamento e agregação.

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace DemoLinqIntro
{
    internal static class Program
    {
        private static void Main()
        {
            List<int> numeros = new() { 1, 2, 3, 4, 5, 6, 7, 8 };

            // Sintaxe de consulta
            IEnumerable<int> consulta =
                from n in numeros
                where n % 2 == 0
                orderby n descending
                select n * n;

            Console.WriteLine("Resultados da consulta LINQ:");
            foreach (int valor in consulta)
            {
                Console.WriteLine(valor);
            }
        }
    }
}
```

Unidade III

```
        Console.WriteLine("Pressione ENTER para sair.");
        Console.ReadLine();
    }
}
```

Os standard query operators são métodos que recebem funções como parâmetros: por exemplo, Where recebe um Func (um predicado), Select recebe um Func (uma função de projeção) e Aggregate recebe um acumulador Func. Em C#, a forma concisa e idiomática

de fornecer essas funções é por meio de expressões lambda, definidas na referência da linguagem como o mecanismo sintático para criar funções anônimas com o operador `=>`.

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace DemoLinqLambda
{
    internal static class Program
    {
        private static void Main()
        {
            List<int> numeros = new() { 1, 2, 3, 4, 5, 6, 7, 8 };

            IEnumerable<int> consulta = numeros
                .Where(n => n % 2 == 0)           // Lambda como predicado
                .OrderByDescending(n => n)        // Lambda como chave de
ordenação
                .Select(n => n * n);              // Lambda como projeção

            Console.WriteLine("Resultados da consulta LINQ com lambdas:");
            foreach (int valor in consulta)
            {
                Console.WriteLine(valor);
            }

            Console.WriteLine("Pressione ENTER para sair.");
            Console.ReadLine();
        }
    }
}
```

Aqui, cada ocorrência de `n => ...` é uma expressão lambda. O compilador, seguindo as regras descritas na documentação, infere os tipos de parâmetro e retorno a partir da assinatura do método de extensão chamado (`Where`, `OrderByDescending`, `Select`) e converte cada lambda em um delegate `Func<...>` compatível. Dessa forma, as expressões lambda são o meio pelo qual se passa lógica de consulta para os operadores LINQ sem precisar declarar métodos nomeados separados.

No nível da linguagem, há duas formas básicas. A primeira é a *expression lambda*, cujo corpo é uma única expressão e cujo valor é o resultado retornado, por exemplo, `(x, y) => x + y`. A segunda é a *statement lambda*, na qual o corpo é um bloco entre chaves, com múltiplas instruções e, se necessário, uma instrução `return`, como em `(x, y) => { var soma = x + y; return soma; }`. Em ambos os casos, a regra é a mesma: declarar a lista de parâmetros entre parênteses, seguida de `=>` e do corpo.

```
// Arquivo Program.cs
using System;

namespace LambdaDemoBasico
{
    internal static class Program
    {
        private static void Main(string[] args)
        {
            // Lambda que calcula o quadrado de um inteiro
            Func<int, int> quadrado = x => x * x;
        }
    }
}
```

PROGRAMAÇÃO ORIENTADA A OBJETOS COM C#

```
Console.WriteLine("Informe um número inteiro: ");
string? entrada = Console.ReadLine();

if (!int.TryParse(entrada, out int valor))
{
    Console.WriteLine("Valor inválido.");
    return;
}

int resultado = quadrado(valor);
Console.WriteLine($"O quadrado de {valor} é {resultado}.");

Console.WriteLine("Pressione ENTER para sair.");
Console.ReadLine();
}
```

Expressões lambda aparecem o tempo todo em LINQ, porque os standard query operators (métodos como Where, Select, OrderBy, Aggregate) são declarados como métodos de extensão que recebem funções como parâmetros, geralmente escritas como lambdas.


```
// Arquivo Program.cs
using System;
using System.Collections.Generic;
using System.Linq;

namespace LambdaDemoLinq
{
    internal static class Program
    {
        private static void Main(string[] args)
        {
            List<int> numeros = new()
            {
                5, 12, 7, 3, 18, 21, 2, 10
            };

            // Lambda usada como predicado em Where
            IEnumerable<int> paresMaioresQueDez =
                numeros.Where(n => n % 2 == 0 && n > 10);

            // Lambda usada como projeção em Select
            IEnumerable<string> descricoes =
                paresMaioresQueDez.Select(n => $"Par filtrado: {n}");

            Console.WriteLine("Números originais:");
            Console.WriteLine(string.Join(", ", numeros));
        }
    }
}
```

Unidade III

```
Console.WriteLine();
Console.WriteLine("Pares > 10 (filtrados com Where):");
Console.WriteLine(string.Join(", ", paresMaioresQueDez));

Console.WriteLine();
Console.WriteLine("Descrições (projetadas com Select):");
foreach (string texto in descricoes)
{
    Console.WriteLine(texto);
}

Console.WriteLine();
Console.WriteLine("Pressione ENTER para sair.");
Console.ReadLine();
}
}
```

Nesse código, `n => n % 2 == 0 && n > 10` é um predicado que o método `Where` recebe como argumento, e `n => $"Par filtrado: {n}"` é uma função de projeção que `Select` usa para transformar cada inteiro em uma string de saída. Veremos detalhes sobre esses operadores em breve.

Outro aspecto central das expressões lambda em C# é a captura de variáveis externas, conhecida como closure. A documentação explica que uma lambda pode referir-se a variáveis que estavam no escopo do método que a definiu; essas variáveis são chamadas de variáveis externas e, quando capturadas, têm seu tempo de vida estendido para que o delegate resultante possa acessá-las com segurança. Isso permite escrever exemplos como o que vemos a seguir, em que uma variável local define um limite que a lambda utiliza:

```

using System;
using System.Collections.Generic;
using System.Linq;

namespace LambdaDemoClosure
{
    internal static class Program
    {
        private static void Main(string[] args)
        {
            Console.WriteLine("Informe um limite mínimo (inteiro): ");
            string? entrada = Console.ReadLine();

            if (!int.TryParse(entrada, out int limite))
            {
                Console.WriteLine("Valor inválido.");
                return;
            }

            List<int> valores = new() { 3, 7, 10, 15, 22, 5, 30 };

            // A lambda captura a variável 'limite' do escopo externo
            Func<int, bool> maiorOuIgualAoLimite = n => n >= limite;

```

52

PROGRAMAÇÃO ORIENTADA A OBJETOS COM C#

```

        var filtrados = valores.Where(maiorOuIgualAoLimite).ToList();

        Console.WriteLine();
        Console.WriteLine($"Valores originais: {string.Join(", ",
valores)}");
        Console.WriteLine($"Valores >= {limite}: {string.Join(", ",
filtrados)}");

        Console.WriteLine();
        Console.WriteLine("Pressione ENTER para sair.");
        Console.ReadLine();
    }
}

```

A variável `limite` é declarada no corpo de `Main`, mas o lambda `n => n >= limite` continua a utilizá-la ao longo da execução da consulta LINQ.

O LINQ foi concebido para ter vários provedores diferentes, todos usando a mesma sintaxe de consulta, mas conectados a fontes de dados distintas. As principais variações são: LINQ to Objects, LINQ to SQL, LINQ to Entities, LINQ to XML e LINQ to DataSet. LINQ to Objects é o caso básico em que se consultam coleções que implementam `IEnumerable`, como `List` ou arrays; a consulta é executada localmente, no CLR, usando delegates comuns.

LINQ to SQL e LINQ to Entities, por sua vez, usam LINQ para acessar dados relacionais. No primeiro caso, o provedor converte a expressão LINQ em SQL para SQL Server.

A sintaxe de consulta é uniforme entre os diversos provedores, de forma que o compilador traduz expressões com palavras-chave como `from`, `where`, `select`, `join` e `orderby` em chamadas a operadores padrão de consulta, e essa tradução independe de o alvo ser LINQ to Objects, LINQ to SQL, LINQ to Entities, LINQ to XML ou LINQ to DataSet. Para o

programador C#, isso se traduz no fato de que o conjunto de palavras-chave usadas em expressões de consulta é o mesmo em todas essas variantes e de que a sintaxe alternativa em forma de métodos, com lambdas em chamadas como `colecacao.Where(x => ...)`, usa os mesmos operadores definidos nas classes `Enumerable` e `Queryable`, que atuam sobre sequências que implementam `IEnumerable` ou `IQueryable`.

6.1 Where/Select/GroupBy/Any, projeções e diferimento de execução

O LINQ permite escrever consultas sobre coleções de dados de forma concisa e fortemente tipada, como se fossem parte da própria linguagem. Em LINQ, operadores de consulta padrão (query operators) criam novas sequências a partir das origens de dados originais, sem alterar os objetos fonte.

Esses operadores podem ser encadeados fluentemente, de modo que os dados fluem da esquerda para a direita através da cadeia. Por exemplo, os operadores `Where`, `OrderBy` e `Select` podem ser combinados para filtrar, ordenar e projetar elementos, sempre retornando novas coleções enquanto preservam a fonte original.

```
int[] numeros = { 1, 2, 3, 4, 5, 6 };  
var pares = numeros.Where(n => n % 2 == 0);
```

```
foreach (var n in pares) Console.WriteLine(n); // Saída esperada: 2 4 6
```

O operador `Select` do LINQ serve para projeção, ou seja, transformação dos elementos de uma sequência em outra forma ou tipo. Cada elemento de entrada é mapeado para um elemento de saída por meio de uma expressão lambda passada a `Select`.

```
string[] nomes = { "ana", "beatriz", "carlos" };  
var maiusculos = nomes.Select(n => n.ToUpper());  
foreach (var n in maiusculos) Console.WriteLine(n);  
// Saída esperada: ANA, BEATRIZ, CARLOS
```

Os operadores `Where`, `Select` e outros do LINQ podem ser encadeados. Por exemplo, imagine filtrar um conjunto de strings para aquelas que contêm a letra "a" e, em seguida, converter o resultado para maiúsculas.

```
string[] nomes = { "Tom", "Dick", "Harry", "Mary", "Jay" };  
var query = nomes .Where(n => n.Contains("a")) .Select(n => n.ToUpper());  
foreach (var nome in query) Console.WriteLine(nome + " ");  
// Exemplo de saída: "HARRY MARY JAY"
```

Além dos operadores de filtragem e projeção, o LINQ oferece o poderoso operador de agrupamento `GroupBy`, que organiza sequências em grupos baseados em uma chave comum. Ele organiza uma sequência de entrada plana em sequências de grupos. Por exemplo, suponha um array de números e considere que desejemos agrupar os números pares e ímpares separadamente:

```
int[] numeros = { 1, 2, 3, 4, 5, 6 };
var grupos = numeros.GroupBy(n => (n % 2 == 0) ? "Par" : "Ímpar");
foreach (var grupo in grupos) {
    Console.WriteLine("Grupo: " + grupo.Key);
    foreach (var n in grupo)
        Console.WriteLine("  " + n);
}
// Saída esperada:
// Grupo: Ímpar
// 1
// 3
// 5
// Grupo: Par
// 2
// 4
// 6
```

Para consultas que precisam simplesmente verificar a existência de elementos, o LINQ fornece o operador quantificador `Any`. Ele retorna `true` se qualquer elemento na sequência satisfaz a determinada condição. Por exemplo:

```
int[] valores = { 2, 3, 4 };
bool existeTres = valores.Any(n => n == 3); // true
bool maiorQueDez = valores.Any(n => n > 10); // false

bool algumPar = new int[] { }.Any(n => n % 2 == 0); // false (sequência vazia)
bool algum = new List().Any(); // false (nenhum elemento)
```

Um ponto fundamental em LINQ é o diferimento de execução das consultas. Isso significa que os operadores padrão de consulta não executam a operação no momento em que são chamados, mas somente quando a sequência é efetivamente enumerada. Veja a seguir:

```
var lista = new List<int>();
lista.Add(1);
// Monta a consulta, mas ainda não a executa
var query = lista.Select(n => n * 10);
lista.Add(2); // Altera a fonte APÓS montar a consulta

// A execução ocorre aqui, no foreach
foreach (int n in query)
    Console.WriteLine(n + "|");
// Saída: 10|20|
```

6.2 IEnumerable x IQueryable (noções para acesso a dados)

Em LINQ, qualquer sequência que implemente IEnumerable pode ser consultada com a mesma sintaxe de consulta típica (por exemplo, usando Where, Select etc.), e da mesma forma qualquer fonte de dados que implemente IQueryable também aceita essas operações de consulta. Em outras palavras, o LINQ foi concebido para fornecer uma sintaxe comum para consultar dados em uma coleção, um documento XML, um banco de dados ou qualquer tipo que suporte as interfaces IEnumerable ou IQueryable.

Tecnicamente, IEnumerable e IQueryable têm propósitos distintos. A interface genérica IEnumerable representa uma sequência de elementos sobre a qual se pode iterar (por exemplo, com foreach). As consultas LINQ sobre objetos que implementam IEnumerable (ou seja, LINQ to Objects) são executadas localmente, em memória, e os operadores padrão correspondentes são encontrados na classe estática Enumerable. Isso significa que as expressões lambda usadas em consultas sobre IEnumerable são convertidas em delegates que são executados diretamente no código IL do programa. Por exemplo, ao aplicar

```
List lista = ObterListaProdutos();  
var caros = lista.Where(p => p.Preco > 100.0m);
```

Já IQueryable estende IEnumerable com funcionalidades adicionais para representar consultas descritas como árvores de expressão (expression trees). Quando se realiza uma consulta LINQ sobre uma fonte de dados remota (por exemplo, uma tabela de banco de dados por meio do Entity Framework), a fonte implementa IQueryable e os operadores padrão estão na classe estática Queryable. Nessa abordagem, em vez de executar o predicado como código local, o LINQ constrói uma árvore de expressão que representa a consulta. Essa árvore é então interpretada pelo provedor de consulta (por exemplo, o provedor do Entity Framework) para gerar uma instrução SQL equivalente, que será executada no banco de dados.

Consultas interpretadas operam sobre sequências que implementam IQueryable<>, e elas resolvem para os operadores de consulta na classe Queryable, os quais emitem árvores de expressão que são interpretadas em tempo de execução. Esse mecanismo permite que o filtro ou a projeção definidos no código C# sejam traduzidos em consultas SQL executadas no servidor de dados, trazendo somente os resultados necessários.

```
var clientesDoBrasil = contexto.Clientes  
.Where(c => c.Pais == "BR" && c.Ativo == true);
```

Nesse caso, contexto.Clientes é um IQueryable (geralmente um DbSet do Entity Framework). A expressão lambda no Where não filtra imediatamente em memória; em vez disso, o provedor LINQ to Entities constrói uma expressão que será convertida em algo como `SELECT * FROM Clientes WHERE Pais = 'BR' AND Ativo = 1` no banco de dados. Assim, a execução ocorre no banco, retornando apenas os clientes brasileiros ativos, o que pode ser muito mais eficiente.

Em termos de desempenho e cenários de uso, IQueryable costuma ser preferível quando se trabalha com fontes de dados externas ou muito volumosas, pois permite que o próprio banco de dados execute parte do trabalho de filtro e agrupamento. Já IEnumerable é adequado para coleções já carregadas em memória ou para cenários em que não há suporte a consultas traduzíveis (por exemplo, em alguns serviços web ou em estruturas sem provedor de consulta).

```
// Fonte de dados local: List<T> implementa IEnumerable<T>
List<Pedido> todosPedidos = Repositorio.ObterPedidos();
IEnumerable<Pedido> pedidosRecentes =
    todosPedidos.Where(p => p.Data > DateTime.Today.AddMonths(-1));

// Fonte de dados remota: DbSet<T> implementa IQueryable<T>
var pedidosEF = contexto.Pedidos; // IQueryable<Pedido>
IQueryable<Pedido> pedidosEFRecentes =
    pedidosEF.Where(p => p.Data > DateTime.Today.AddMonths(-1));
```

unidade IV

7 ERROS E TESTES

Os chamados erros de compilação surgem quando o código viola regras da linguagem ou do sistema de tipos: um identificador inexistente, uma conversão impossível, uma instrução malformada etc. O compilador do C# analisa o código-fonte, verifica a conformidade com a especificação da linguagem e, se encontrar um problema, não gera o executável.

Exceções, objetos derivados de System.Exception que representam falhas detectadas durante a execução do programa. Ao acessar um elemento fora dos limites de um array, o runtime lança IndexOutOfRangeException; ao desreferenciar uma variável nula, surge NullReferenceException; ao tentar dividir por zero, aparece DivideByZeroException; problemas de entrada/saída (I/O, do inglês input/output) típicos resultam em subclasses de IOException.

exceções, objetos derivados de System.Exception que representam falhas detectadas durante a execução do programa. Ao acessar um elemento fora dos limites de um array, o runtime lança IndexOutOfRangeException; ao desreferenciar uma variável nula, surge NullReferenceException; ao tentar dividir por zero, aparece DivideByZeroException; problemas de entrada/saída (I/O, do inglês input/output) típicos resultam em subclasses de IOException.

try, catch e finally. Um trecho de código que pode falhar é envolvido em try; se alguma instrução ali lançar uma exceção, o runtime interrompe o fluxo normal, percorre a pilha até encontrar um catch compatível com o tipo da exceção e transfere o controle para esse ponto.

Dentro do catch, o desenvolvedor pode registrar logs, notificar camadas superiores, traduzir a exceção em um tipo mais aderente ao domínio da aplicação ou simplesmente repropagar o problema com throw.

O bloco finally, quando presente, é executado independentemente de ter ocorrido ou não uma exceção e costuma ser usado para liberar recursos. Versões modernas da linguagem acrescentam filtros de exceção com a palavra-chave when, permitindo que um catch só seja considerado se uma condição booleana adicional for verdadeira.

Entre as ferramentas de testes para C# e .NET, o framework xUnit ocupa lugar de destaque, sendo o mais popular no ecossistema moderno. Diferentemente de abordagens mais antigas, como MSTest, o xUnit foi desenvolvido especificamente para .NET Core e .NET 5/6/7/8, favorecendo simplicidade e extensibilidade. Em projetos .NET, a utilização do xUnit é praticamente padrão para testes de integração e unidade. A filosofia do xUnit se alinha a práticas ágeis, de modo que os testes são tratados como código de primeira classe, escritos da mesma forma que o código de produção.

7.1 Exceções, hierarquia, tratamento e criação de exceções

O tratamento de erros e condições excepcionais é realizado por meio de exceções, que são objetos especiais lançados quando o programa encontra problemas em tempo de execução.

O mecanismo de exceções separa claramente o código de identificação de erros do código de tratamento de erros. Isso significa que um bloco de código pode simplesmente lançar (throw) uma exceção quando ocorrer uma situação inesperada, e outro trecho de código (geralmente em um try/catch) se encarrega de capturá-la e tratá-la

O código propenso a erros é colocado no bloco try, enquanto um ou mais blocos catch seguintes tratam exceções lançadas. Opcionalmente, um bloco finally executa sempre, ocorrendo exceção ou não, sendo ideal para liberar recursos (arquivos, conexões etc.).

```
try
{
    // Código que pode lançar exceções
    int resultado = Numerador / Denominador;
    Arquivo.Write("Salvando dados...");
}
catch (DivideByZeroException ex)
{
    // Tratamento específico de divisão por zero
    Console.WriteLine($"Erro de divisão: {ex.Message}");
}
catch (IOException ex)
{
    // Tratamento de erros de I/O
    Console.WriteLine($"Erro de I/O: {ex.Message}");
}
catch (Exception ex)
{
    // Tratamento genérico de outras exceções
    Console.WriteLine($"Erro inesperado: {ex.Message}");
}
finally
{
    // Código sempre executado (por exemplo, liberar recursos)
    Console.WriteLine("Operação finalizada.");
}
```

Cada tipo de exceção (DivideByZeroException, IOException etc.) é capturado por seu bloco catch específico, evitando-se assim o uso de um único catch (Exception) genérico de modo desnecessário. O bloco finally mostra um uso típico: independentemente de erro ou não, a mensagem de finalização é exibida, representando operações como fechar arquivos ou liberar memória. Essa separação permite tratar cada situação de erro adequadamente.

Cláusula when, que permite testar condições adicionais no momento da captura. Veja a seguir:

```
try
{
    ProcessarArquivo(caminhoArquivo);
}
catch (FileNotFoundException ex) when (caminhoArquivo.EndsWith(".txt"))
{
    Console.WriteLine("Arquivo de texto não encontrado. Crie o arquivo antes de executar.");
}
catch (FileNotFoundException ex)
{
    Console.WriteLine($"Arquivo não encontrado: {ex.FileName}");
}
```

O primeiro catch utiliza um filter (when) para tratar especificamente a ausência de arquivos de texto (.txt) de forma personalizada; o segundo bloco captura outras falhas de arquivo padrão.

Além das exceções predefinidas pelo .NET, os desenvolvedores podem criar exceções personalizadas derivadas de Exception (ou ApplicationException). Essas classes customizadas permitem sinalizar erros específicos do domínio da aplicação. Por exemplo:

```
// Definição de exceção personalizada
public class UsuarioInvalidoException : Exception
{
    public UsuarioInvalidoException(string mensagem)
        : base(mensagem) { }

    public UsuarioInvalidoException(string mensagem, Exception innerException)
        : base(mensagem, innerException) { }
}

// Lançando a exceção personalizada
if (usuario == null)
    throw new UsuarioInvalidoException("O usuário não pode ser nulo.");
```

UsuarioInvalidoException herda de Exception e repassa a mensagem de erro à classe base.


```

try
{
    ConectarBanco();
}
catch (Exception ex)
{
    Console.WriteLine($"Erro: {ex.Message}");
    Console.WriteLine($"Origem: {ex.StackTrace}");
    if (ex.InnerException != null)
    {
        Console.WriteLine($"Erro interno: {ex.InnerException.Message}");
    }
}

```

Vimos que a cláusula `when` introduz filtros de exceção no C# e permite avaliar uma condição extra no `catch`, sem precisar duplicar o bloco. Em essência, o filtro garante que um `catch` só seja executado se a expressão após `when` for verdadeira.

7.2 Testes com xUnit e uso do Gerenciador de Testes

O framework xUnit.net é uma das ferramentas de testes de unidade. Cada método de teste é marcado com atributos especiais: `[Fact]` para testes sem parâmetros fixos (assertivas invariantes) e `[Theory]` para testes parametrizados com dados de entrada diversos. No desenvolvimento orientado a testes (TDD, do inglês *test-driven development*), por exemplo, começa-se escrevendo um teste que falha e depois se implementa o código de produção para fazê-lo passar.

A integração do xUnit com o Visual Studio facilita esse fluxo: ao criar um projeto de teste, geralmente se usa o comando `dotnet new xunit` ou o assistente da IDE, que já adiciona automaticamente referências essenciais, como `xunit`, `xunit.runner.visualstudio` e `Microsoft.NET.Test.Sdk`.

Em um arquivo de projeto típico (por exemplo, o `.csproj` de um projeto de testes xUnit), veremos trechos como:

```

<ItemGroup>
  <PackageReference Include="Microsoft.NET.Test.Sdk" Version="..." />
  <PackageReference Include="xunit" Version="..." />
  <PackageReference Include="xunit.runner.visualstudio" Version="..." />
</ItemGroup>

```

Cada método de teste recebe o atributo `[Fact]` quando representa um cenário isolado, ou `[Theory]` acompanhado de atributos `[InlineData]` para testar múltiplos casos.

```

public class CalculadoraTests
{
    [Fact]
    public void Somar_2e2_Retorna4()
    {
        int resultado = Calculadora.Somar(2, 2);
        Assert.Equal(4, resultado);
    }
}

```

O atributo **[Fact]** indica ao runner que o método é um teste que deve passar invariavelmente. Para validar comportamentos com vários valores, usa-se **[Theory]** e múltiplos **[InlineData]**. Ao testar números ímpares, teríamos o que segue:

```
public class NumeroTests
{
    [Theory]
    [InlineData(3)]
    [InlineData(4)]
    [InlineData(5)]
    public void EhImpar_DeveRetornarTrueApenasParaImpares(int valor)
    {
        bool ehImpar = (valor % 2) == 1;
        Assert.True(ehImpar == (valor % 2 == 1));
    }
}
```

8 MEMÓRIA E CONCORRÊNCIA INTRODUTÓRIA

Muitas aplicações modernas são dominadas por operações I/O-bound, nas quais o gargalo não está no cálculo local, mas na espera por respostas de rede, disco ou outros serviços externos; se essas esperas forem tratadas de modo bloqueante, a interface gráfica congela, servidores ficam incapazes de atender a novas requisições e a percepção de qualidade do software cai rapidamente.

O modelo baseado em Tasks permite expressar essas operações como trabalhos independentes, passíveis de composição, enquanto a sintaxe `async/await` oferece uma forma mais direta de escrever fluxos assíncronos. A introdução do cancelamento cooperativo, por sua vez, responde à necessidade de interromper operações longas sem deixar recursos pendurados e sem recorrer a mecanismos abruptos.

Assim, este tópico se insere em um conjunto de preocupações que vão da alocação de memória ao controle refinado de concorrência, fornecendo ao leitor o pano de fundo conceitual necessário para compreender por que essas construções de C# são centrais no desenvolvimento profissional.

8.1 Valor x referência, GC, struct x class

A linguagem C# executa sobre o ambiente gerenciado do .NET, que provê gerenciamento automático de memória por meio de um coletor de lixo (garbage collector, GC).

Cada chamada a `new` calcula o tamanho dos campos do objeto, adiciona o overhead (por exemplo, ponteiro de tipo e índice de bloco de sincronização) e então, se houver espaço suficiente na região atual do heap, aloca o objeto, zera a memória, invoca o construtor e ajusta o ponteiro `NextObjPtr` para o próximo espaço livre. CLR emprega o GC para deletar objetos inacessíveis.

O CLR concentra-se apenas em variáveis de tipos por referência, já que somente elas podem apontar para objetos no heap; variáveis de tipos por valor contêm a instância diretamente e não são consideradas raízes.



Lembrete

Uma classe é um tipo por referência; uma struct é um tipo por valor e, consequentemente, structs representam tipos simples, com semântica de valor, nos quais a atribuição copia efetivamente o valor.

Considere o seguinte código ilustrativo:

```
int a = 10;
int b = a;
a = 20;
// a vale 20, b continua 10 (tipos por valor).
class Pessoa { public int Id; }
Pessoa p1 = new Pessoa { Id = 1 };
Pessoa p2 = p1;
p1.Id = 2;
// p1.Id e p2.Id agora valem 2, pois p1 e p2 referem o mesmo objeto.
```

Structs são preferencialmente usados para tipos imutáveis ou pequenos, já que evitam sobrecarga de alocação no heap e coleta de lixo. Classes, por outro lado, suportam herança e são usadas para objetos mais complexos, é comum usar structs para representar pontos, retângulos, datas e pequenos dados com comportamento de valor, enquanto as classes encapsulam a lógica de negócio e os estados mais complexos.

8.2 Task, async/await, cancelamento com CancellationToken

A programação concorrente em C# utiliza rotinas de alta abstração chamadas Tasks (tarefas).

Em vez de manipular diretamente threads, o .NET introduziu a Task Parallel Library (TPL), na qual cada Task corresponde a uma unidade de trabalho distribuída pelo thread pool.

Threads, no contexto de sistemas operacionais e de C#, são a linha de execução que de fato consome CPU dentro de um processo. Em termos clássicos, Silberschatz, Galvin e Gagne (2018) definem thread como a unidade básica de utilização da CPU: ela possui um identificador próprio, um contador de programa, um conjunto de registradores e uma pilha, compartilhando com outras threads do mesmo processo o código, os dados e demais recursos do sistema operacional, como arquivos abertos e sinais, dentro de um único processo, é possível ter várias threads executando em paralelo (realmente paralelas em máquinas multicore, ou intercaladas pelo escalonador em máquinas de um único núcleo), cada uma com seu fluxo de instruções, embora todas enxerguem o mesmo espaço de endereçamento do processo.

O processo é o contêiner pesado que define o espaço de endereçamento, os handles de arquivo, as conexões de rede e demais recursos globais. As threads são as entidades leves que o sistema operacional agenda para executar instruções nesse espaço.

Em .NET e C#, essa ideia é preservada, mas aparece com uma camada de abstração adicional. O CLR expõe threads gerenciadas, representadas pela classe `System.Threading.Thread`, que normalmente mapeiam para threads do sistema operacional.

Diz-se que uma operação é I/O-bound quando o tempo total de execução é dominado pela espera por entrada/saída (rede, disco, sistema de arquivos, soquete etc.) e não pelo processamento de CPU. Em uma chamada HTTP, por exemplo, a maior parte do tempo é gasta aguardando o servidor responder e os dados viajarem pela rede. Se essa espera for feita de forma síncrona, a thread fica bloqueada, sem poder processar outras mensagens da interface ou outras requisições.

Em cenários práticos, muitas operações de I/O (como chamadas web, acesso a banco ou leitura de arquivos) expõem APIs assíncronas nativas que retornam diretamente `Task` ou `Task<T>`. O método `HttpClient.GetStringAsync(uri)`, por exemplo, inicia uma requisição HTTP GET para o endereço indicado e devolve imediatamente uma `Task`.

Essa `Task<string>` representa uma operação em andamento que, no futuro, produzirá uma string com o corpo da resposta.

conclusão. A seguir, temos um trecho comum em aplicações .NET, que inicia uma tarefa longa e depois aguarda sem travar a aplicação:

```
public async Task CalcularOperacaoDemoradaAsync() {  
    // Inicia uma tarefa que simula operação custosa  
    await Task.Delay(2000); // Espera 2 segundos de forma assíncrona  
    Console.WriteLine("Operação demorada concluída");  
}
```

O modelo task-based de programação assíncrona em C# (TAP) fornece uma base rica para lidar com concorrência e I/O assíncrono de forma expressiva. Tarefas (`Task` e `Task<T>`) representam unidades de trabalho que podem ser lançadas ao thread pool ou a provedores de I/O assíncrono, e seu resultado pode ser obtido com `await` sem bloquear a thread de chamada.

O uso de `async/await` simplifica a programação assíncrona, mantendo o código linear e fácil de entender. Além disso, práticas de cancelamento cooperativo (usando `CancellationToken`) permitem que aplicativos permaneçam responsivos e graciosos diante de solicitações de cancelamento, conforme recomendado por Cleary (2019), que discute amplamente o uso de `Tasks` e cancelamento cooperativo em C#.